



Univerzitet u Nišu  
Elektronski fakultet u Nišu



## Storage engine i upravljanje podacima u PostgreSQL bazi podataka

Seminarski rad

Mentor:  
Prof. dr Aleksandar Stanimirović

Student:  
Luka Veličković, 2101

Niš, April 2026.

# Sadržaj

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1 Uvod.....                                                       | 4  |
| 2 PostgreSQL Data Directory i organizacija podataka na disku..... | 5  |
| 2.1 base/ direktorijum.....                                       | 5  |
| 2.2 global/ direktorijum.....                                     | 5  |
| 2.3 pg_commit_ts/ direktorijum.....                               | 6  |
| 2.4 pg_xact/ direktorijum.....                                    | 6  |
| 2.5 pg_wal/ direktorijum.....                                     | 6  |
| 2.6 Drugi direktorijumi.....                                      | 6  |
| 2.7 Templates.....                                                | 7  |
| 3 PostgreSQL storage engine osnove.....                           | 8  |
| 3.1 PostgreSQL stranice.....                                      | 8  |
| 3.2 Heap storage.....                                             | 10 |
| 3.3 MVCC.....                                                     | 11 |
| 3.3.1 Heap-Only Tuples.....                                       | 12 |
| 3.4 Praktični prikaz.....                                         | 13 |
| 4 PostgreSQL write path i read path.....                          | 16 |
| 4.1 Arhitektura.....                                              | 16 |
| 4.1.1 Lokalna memorija.....                                       | 16 |
| 4.1.2 Shared memory.....                                          | 17 |
| 4.1.3 Disk storage.....                                           | 18 |
| 4.2 Write path.....                                               | 18 |
| 4.2.1 Potvrda commit-a.....                                       | 19 |
| 4.2.2 Optimizacija potvrde commit-a.....                          | 20 |
| 4.2.2.1 Grupni commit.....                                        | 20 |
| 4.2.2.2 Batch transakcije.....                                    | 20 |
| 4.2.2.3 COPY.....                                                 | 20 |
| 4.2.2.4 Unlogged tables.....                                      | 20 |
| 4.3 Read path.....                                                | 21 |
| 4.4 Praktični prikaz.....                                         | 22 |
| 4.4.1 Write path.....                                             | 22 |
| 4.4.2 Optimizacija potvrde commit-a.....                          | 24 |
| 4.4.2.1 Grupni commit.....                                        | 24 |
| 4.4.2.2 Commit after insert VS batch insert VS copy.....          | 26 |
| 4.4.3 Read path.....                                              | 26 |
| 5 Upravljanje baferima.....                                       | 28 |
| 5.1 Osnovni prikaz.....                                           | 28 |
| 5.2 Buffer table.....                                             | 29 |
| 5.3 Buffer descriptor.....                                        | 30 |
| 5.4 Buffer manager lock-ovi.....                                  | 32 |
| 6 WAL.....                                                        | 34 |
| 6.1 Osnovni pojmovi.....                                          | 34 |
| 6.2 Struktura WAL segment fajlova.....                            | 35 |
| 6.3 Struktura XLOG record-a.....                                  | 36 |
| 6.4 Upravljanje WAL-om.....                                       | 38 |

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 6.5 Praktični prikaz.....                                           | 39 |
| 6.5.1 XLOG records.....                                             | 39 |
| 6.5.2 Uticaj checkpoint-a i full-page XLOG zapisa na WAL i I/O..... | 41 |
| 7 Asinhroni I/O.....                                                | 43 |
| 7.1 Asinhroni i direktni I/O.....                                   | 43 |
| 7.2 io_uring.....                                                   | 44 |
| 7.3 Asinhroni I/O u PostgreSQL-u.....                               | 45 |
| 7.4 Praktični prikaz.....                                           | 46 |
| 8 Literatura.....                                                   | 48 |

# 1 Uvod

Perzistentno skladištenje podataka predstavlja neizostavni deo bilo koje aplikacije. U zavisnosti od potreba aplikacije mogu se koristiti *flat* fajlovi, koji mogu biti tekstualni ili binarni. Aplikacije koje svoje podatke skladište na ovakav način su najčešće *offline* aplikacije koje se izvršavaju lokalno na korisnikovom ličnom računaru, npr. aplikacije za editovanje teksta, kao što su *Microsoft Word* ili *Notepad*. Kod aplikacija koje uključuju komunikaciju između klijenta i servera, podaci se na serveru najčešće skladište na strukturiran način korišćenjem baze podataka. U osnovi, baza podataka predstavlja kolekciju fajlova, u okviru kojih se podaci smeštaju na određen način. Korisniku baza podataka obezbeđuje nivo apstrakcije koji dozvoljava upravljanje podacima bez potrebe za poznavanjem načina na koji su podaci fizički smešteni u fajlove, ili čak kako su fizički raspoređeni po različitim čvorovima u slučaju distribuirane baze podataka.

U svetu baza podataka, način na koji su podaci organizovani iz perspektive korisnika naziva se logička struktura baze podataka. Logička struktura uključuje definisanje tabela, pogleda, indeksa... Korisnik koristi SQL deklarativni jezik kako bi definisao strukturu baze podataka za svoje potrebe, bez zalaženja u detalje kako će definisane strukture biti kreirane i skladištene. Baza podataka je u tom pogledu kompleksan sistem koji krije ogromnu logiku iza jednostavnog SQL interface-a. [1]

U većini slučajeva korišćenje baze podataka ne zahteva poznavanje unutrašnjeg funkcionisanja baze podataka. Čak naprotiv, osnovni cilj postojanja baze podataka je da sakrije kompleksnost manuelnog upravljanja podacima. Ipak, baza podataka nije magična kutija i iako u pozadini vrši različite optimizacije, ona je efikasna onoliko koliko je efikasan i korisnik koji je koristi. Poznavanje fizičke strukture baze podataka može značajno da doprinese optimalnijem korišćenju baze podataka.

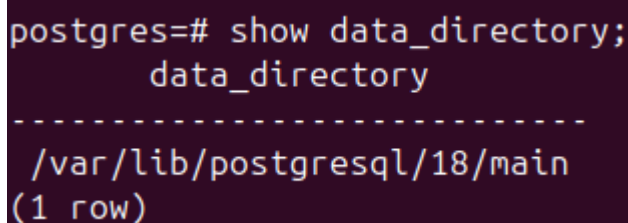
U centru bilo koje baze podataka je podsistem koji upravlja skladištenjem podataka, tzv. *storage engine*. Različiti tipovi baza podataka skladište podatke na različite načine kako bi se različiti načini pristupa podacima optimizovali. *Storage engine* baze podataka koja treba da prihvati veliki broj upisa, a manji broj čitanja će značajno drugačije pristupiti organizaciji podataka od *storage engine-a* baze podataka u koju se podaci prevashodno čitaju. U slučaju relacionih baza podataka *storage engine* je izuzetno složen, obzirom da je korisniku omogućeno da podacima pristupa na bilo koji način, bez ikakvih ograničenja. Kako bi baze podataka obezbedile što efikasniji pristup podacima (upis i čitanje) takođe je neizostavno važan i odnos sa, kao i ograničenja koja nameće operativni sistem. Radi ubrzanja pristupa, *storage engine* se može upustiti u keširanje podataka. Obzirom da relacione baze podataka treba da obezbede *ACID* svojstva uloga *storage engine-a* je utoliko bitnija.

Ovaj rad se fokusira na istraživanju različitih aspekata *storage engine-a* kod PostgreSQL baze podataka. PostgreSQL baza podataka je *open-source* baza podataka koja omogućava promenu i čak vrlo "jednostavan" razvoj custom *storage engine-a*. Najnovija verzija PostgreSQL u vreme pisanja je 18 i jedna od najznačajnijih promena koju je ova verzija donela je novi asinhroni pristup disku. Sve pomenuto čini PostgreSQL vrlo pogodnom bazom podataka za proučavanje upravljanja podacima u relacionim sistemima.

## 2 PostgreSQL Data Directory i organizacija podataka na disku

Pod pojmom *Data Directory* kod relacionih baza podataka se podrazumeva fizička lokacija gde klaster baze podataka skladišti svoje *data* fajlove, logove transakcija i konfiguracione fajlove. U svetu PostgreSQL-a, *Data Directory* se sreće i pod imenom **PGDATA**. U suštini, radi se o direktorijumu koji sadrži sve neophodne fajlove koje PostgreSQL koristi prilikom manipulacije objektima baze.

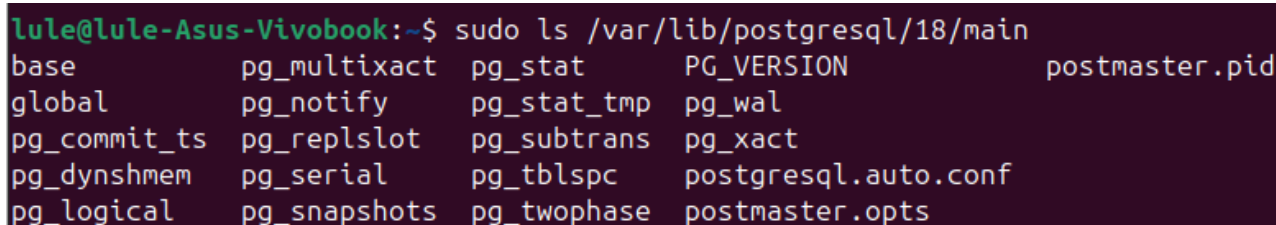
Lokacija ovog direktorijuma može se otkriti korišćenjem `show data_directory` komande. Za potrebe ovog seminarskog rada korišćena je verzija 18 PostgreSQL-a, koja je pokrenuta kao servis na Linuks operativnom sistemu. Rezultat izvršenja pomenute komande:



```
postgres=# show data_directory;
 data_directory
-----
/var/lib/postgresql/18/main
(1 row)
```

Slika 1 - Primer izvršenja `show data_directory` komande

Na slici 2 prikazan je sadržaj ovog direktorijuma.



```
lule@lule-Asus-Vivobook:~$ sudo ls /var/lib/postgresql/18/main
base          pg_multixact  pg_stat       PG_VERSION    postmaster.pid
global        pg_notify     pg_stat_tmp   pg_wal
pg_commit_ts  pg_repslot    pg_subtrans   pg_xact
pg_dynshmem   pg_serial     pg_tblspc     postgresql.auto.conf
pg_logical    pg_snapshots  pg_twophase   postmaster.opts
```

Slika 2 - Sadržaj `$PGDATA` direktorijuma

U nastavku će ukratko biti opisan sadržaj najznačajnijih poddirektorijuma u okviru `PGDATA` direktorijuma.

### 2.1 base/ direktorijum

*Base/* direktorijum je jedan od najvažnijih direktorijuma fizičkog skladišta kod PostgreSQL-a. Ovaj direktorijum je zapravo mesto gde korisnički podaci žive na disku. Svaka PostgreSQL baza podataka u klasteru poseduje svoj poddirektorijum u okviru *base/* direktorijuma, koji sadrži fajlove sa tabelama, indeksima, sekvencama i ostalim povezanim objektima. [1]

Svaka baza podataka u PostgreSQL klasteru ima jedinstveni *Object ID (OID)*. Takođe, svaki objekat u okviru jedne instance baze podataka sadrži jedinstveni *OID*. Svaki objekat skladišti se u posebnom fajlu. Tabele, odnosno objekti, čija veličina prevazilazi određeni limit (najčešće 1GB) se dele u veći broj fajlova. Na primer, ukoliko je *OID* baze podataka 1, a *OID* objekta u bazi 25, lokacija fajla koji opisuje objekat bi bila `$PGDATA/base/1/25`. [1] Format pojedinačnih fajlova biće opisan u nekom od narednih poglavlja.

### 2.2 global/ direktorijum

*Global/* direktorijum sadrži zajedničke objekte na nivou klastera. Pošto su ovi objekti zajednički za sve baze podataka u klasteru, kako bi se obezbedila konzistentnost podataka oni se nalaze u

zajedničkom direktorijumu za sve baze *global/*. Ovaj direktorijum sadrži fajlove koji predstavljaju sistemski katalog PostgreSQL-a, role i permisije svih korisničkih naloga, podešavanja na nivou klastera. [1]

## 2.3 pg\_commit\_ts/ direktorijum

Kod PostgreSQL-a ovaj direktorijum sadrži vremenske oznake (*timestamps*) *commit-ova* kompletiranih transakcija. Ove vremenske oznake predstavljaju tačno vreme kada je transakcija bila komitovana u sistemu. Praćenje ovih vremenskih trenutaka podrazumevano nije uključeno, i od značaja je kod logičke replikacije kako bi se rešavali mogući konflikti. [1]

## 2.4 pg\_xact/ direktorijum

Ovaj direktorijum kod PostgreSQL-a igra ključnu ulogu u praćenju statusa svih transakcija koje se dešavaju u sistemu. Statusi u kojima se može naći jedna transakcija su *in-progress*, *committed* i *rolled back (aborted)*. Ovaj poddirektorijum takođe ima značajnu ulogu i za funkcionisanje MVCC sistema, kako bi MVCC mogao da odredi koji podaci treba da budu vidljivi u okviru koje transakcije. [1]

## 2.5 pg\_wal/ direktorijum

*Pg\_wal/* direktorijum takođe predstavlja jedan od najvažnijih direktorijuma kod PostgreSQL-a. U njemu se skladište WAL (*Write-Ahead Log*) fajlovi, koji su ključni za postizanje *durability* svojstva kod relacionih baza podataka. Uloga WAL-a je oporavak od pada sistema, vraćanje baze podataka u neko od prethodnih stanja, kao i replikacija podataka. WAL fajlovi mogu neograničeno da rastu i kako bi se sprečilo nekontrolisani rast potrebno je da se povremeno arhiviraju ili da se recikliraju. [1]

## 2.6 Drugi direktorijumi

Pored ovih, kao što se može primetiti na slici 2 postoji još puno direktorijuma u okviru *\$PGDATA*. Njihova uloga je specijalizovana za određeni aspekt baze podataka. [1, 3]

1. *pg\_logical/* - koristi se za interno upravljanje logičkom replikacijom

2. *pg\_stats/* - koristi se za skladištenje statistike baze podataka koju *query optimizer* koristi

3. *pg\_tblspc/* - koristi se se za upravljanje *tablespace*-vima. Sadrži *symlinks* kod Linuksa, odnosno *shortcuts* kod Windows-a do fizičke lokacije gde se *tablespace* nalazi. *Tablespace* predstavlja lokaciju u fajl sistemu gde će korisnički podaci biti fizički skladišteni. Ukoliko sistem poseduje veći broj hard diskova, korišćenjem *tablespace*-a moguće je da se korisnički podaci skladište na bržem hard disku. [2]

4. *pg\_dynshmem/* – poddirektorijum koji sadrži fajlove koji koristi podsistem za upravljanje dinamičkom deljivom memorijom

5. *pg\_multixact/* - poddirektorijum koji sadrži statuse multitransakcionih podataka (koristi se za deljive *row-level lock-ove*)

6. *pg\_notify/* - koristi se za implementaciju *LISTEN/NOTIFY* funkcionalnosti

7. *pg\_replslot/* - sadrži informacije o replikacionim slotovima. Koristi se kod logičke replikacije

8. *pg\_serial/* - sadrži informacije o komitovanim *serializable* transakcijama
9. *pg\_snapshots/* - sadrži informacije o eksportovanim *snapshot*-ovima
10. *pg\_stat\_tmp/* - privremeni fajlovi sa statistikama koje koristi *optimizer*
11. *pg\_subtrans/* - informacije o statusima podtransakcija
12. *pg\_twophase/* - koristi se kod *two-phase commit-a*. Sadrži informacije o stanju transakcija.

## 2.7 Templates

U *base/* direktorijumu, pored pojedinačnih poddirektorijuma koji se koriste za korisnički kreirane baze podataka, postoji nekoliko posebnih poddirektorijuma koji su poznati pod nazivom *templates*. Oni igraju ključnu ulogu prilikom kreiranja nove baze podataka:

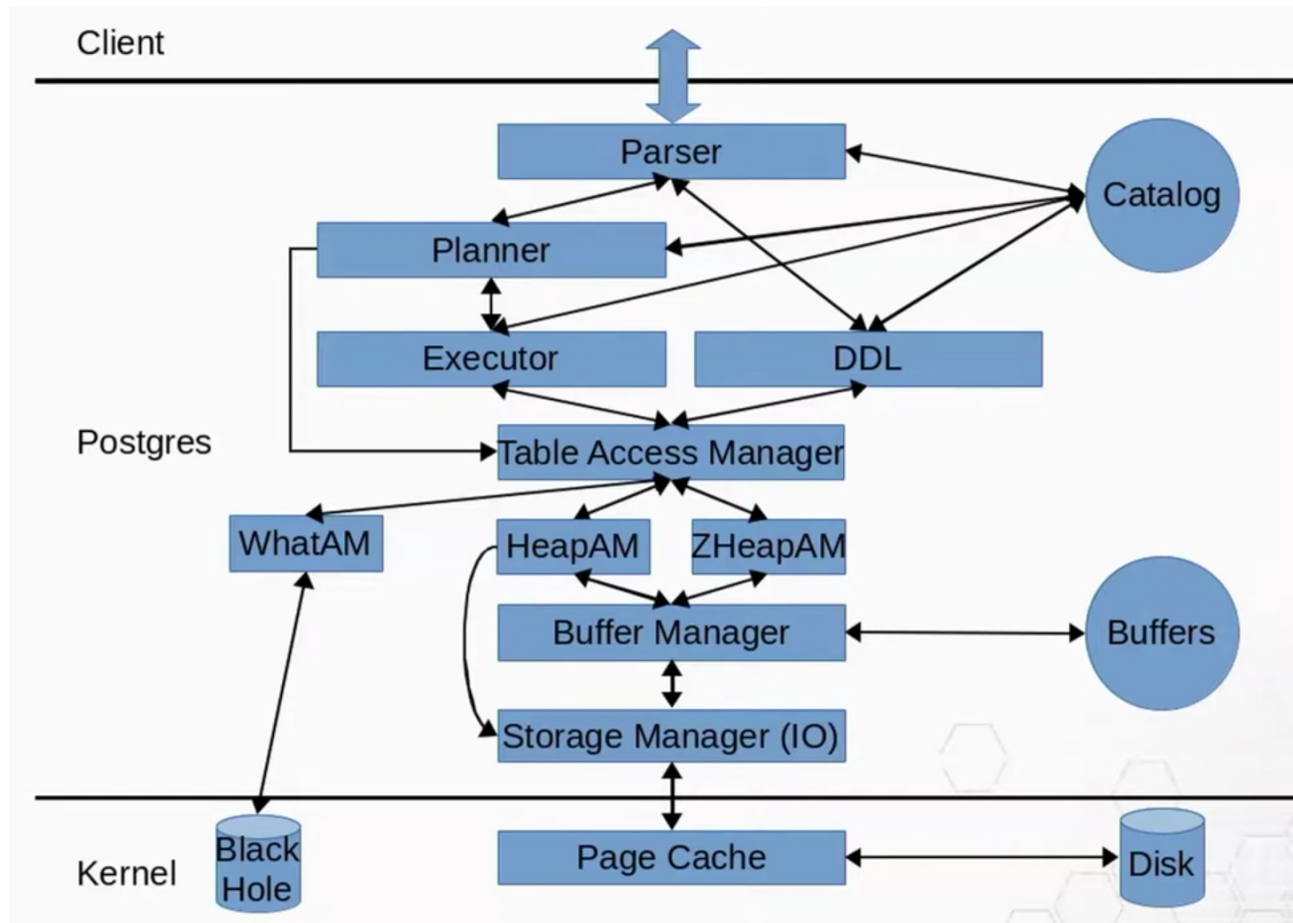
1. *template0* – sadrži originalni sistemski katalog i strukture koje se kreiraju prilikom inicijalizacije PostgreSQL-a. Ne sadrži nikakve korisnički definisane objekte. Ovaj poddirektorijum je read-only. Koristi se prilikom kreiranja nove baze podataka, kada je cilj kreiranje čiste baze podataka, kao da se Postgres prvi put inicijalizuje.

2. *template1* – slično kao *template0*, koristi se prilikom kreiranja nove baze podataka, ali za razliku od *template0*, *template1* nije read-only, što znači da će novokreirana baza podataka naslediti sve definisane objekte koji se nalaze u *template1*.

3. *postgres* – default administrativna baza koja se kreira prilikom instalacije klastera. Uloga *postgres* baze je administracija (podešavanje konfiguracije, upravljanje rolama, testiranje komandi bez uticaja na produkcione podatke, instalacija i verifikacija ekstenzija...). [1]

### 3 PostgreSQL storage engine osnove

Kao što je pomenuto u uvodnom poglavlju ovog seminarskog rada, PostgreSQL *storage engine* je jako komplikovan zbog velikog broja podsistema kojima on upravlja. Na slici 3 prikazana je uprošćena arhitektura PostgreSQL baze podataka, gde je nešto veći akcenat stavljen na *storage* podsistem.



Slika 3 - Arhitektura PostgreSQL baze podataka, preuzeto iz [4]

Komponente koje čine *storage engine* sa slike 3 su: *Table Access Manager*, *Table Access Method* (*HeapAM*, *ZHeapAM*), *Buffer Manager*, *Buffers*, *Storage Manager (IO)*, kao i interakcija sa operativnim sistemom u vidu keširanja stranica i pristup disku.

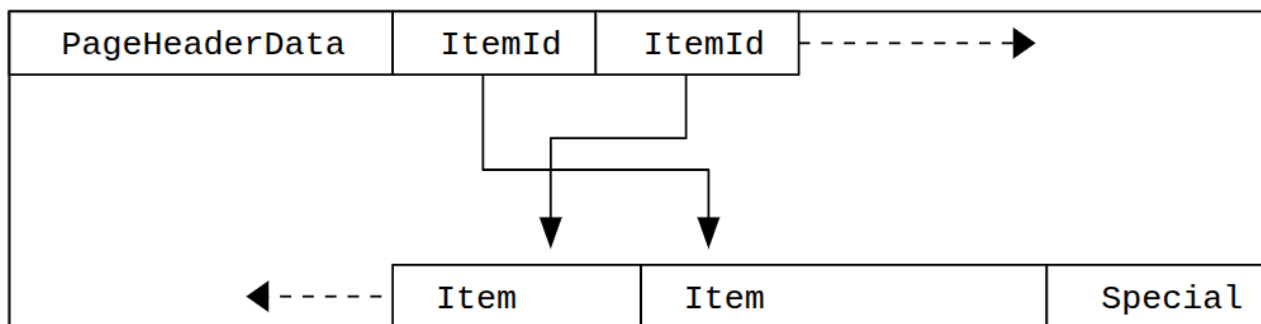
U okviru ovog poglavlja biće opisani osnovni pojmovi koji se odnose na način skladištenja podataka kod PostgreSQL-a, kao što su stranice/blokovi, *heap storage*, *tuples* i uticaj *MVCC-a* na čuvanje podataka. Za upravljanje navedenim strukturama zaduženi su *Table Access Method-i*. U narednim poglavljima biće opisane preostale komponente.

#### 3.1 PostgreSQL stranice

Kod PostgreSQL-a sve tabele i indeksi su skladišteni kao niz stranica (*page*) fiksne dužine (najčešće 8kB). [5] Kod različitih izvora pojam stranice se može naći i pod drugim imenima, najčešće blok. [6] Korišćenje termina blok u ovom kontekstu može biti zbunjujuće, obzirom da se u izvornom kodu PostgreSQL-a definiše blok kao jedinica i/o operacije [7]. Kod tabela, jedan red se može naći u bilo kojoj stranici koja je vezana za tu tabelu, dok je kod indeksa prva stranica



namenjena za čuvanje metapodataka o indeksu. Na slici 4 prikazan je *layout* jedne stranice kod PostgreSQL-a.



Slika 4 - Layout stranice kod PostgreSQL-a, preuzeto iz [5]

Najpre, prvih 24 bajtova stranice čini tzv. *header*. Radi se o delu stranice koji sadrži osnovne informacije o stranici, uključujući pokazivač na slobodni prostor. Na slici 5 prikazan je detaljan sadržaj ovog dela stranice. [5]

| Field               | Type           | Length  |
|---------------------|----------------|---------|
| pd_lsn              | PageXLogRecPtr | 8 bytes |
| pd_checksum         | uint16         | 2 bytes |
| pd_flags            | uint16         | 2 bytes |
| pd_lower            | LocationIndex  | 2 bytes |
| pd_upper            | LocationIndex  | 2 bytes |
| pd_special          | LocationIndex  | 2 bytes |
| pd_pagesize_version | uint16         | 2 bytes |
| pd_prune_xid        | TransactionId  | 4 bytes |

Slika 5 - Sadržaj header-a stranice

- *pd\_lsn* – skladišti *LSN* (log sequence number) poslednjeg *WAL* zapisa, čije su promene reflektovane na stranici. Ovo polje se koristi kako bi se obezbedilo da se uvek zapis u *WAL-u flush-uje* na disk, pre stranice. Kada je potrebno stranicu upisati na disk, prvo se proverava da li je *WAL* zapisan na disk barem do *pd\_lsn-a*. Ukoliko jeste, bezbedno je upisati stranicu na disk, u suprotnom prvo je potrebno upisati *WAL*.
- *pd\_checksum* – *checksum-a* stranice, kako bi bilo moguće proveriti njenu validnost

- *pd\_upper* – ofset do početka slobodnog prostora
- *pd\_lower* – ofset do kraja slobodnog prostora
- *pd\_special* – ofset do početka *special* prostora
- *pd\_pagesize\_version* – informacija o veličini stranice i verziji koja se koristi
- *pd\_prune\_xid* - najstariji *XID* od *heap tuple-ova* koji mogu biti uklonjeni iz stranice
- *pg\_flags* – flagovi koji opisuju status stranice (ima slobodnih, puna...)

Nakon *headera* stranice slede četvorobajtni pokazivači na redove (*tuple-ove*) koji se skladište na stranici. Svaki pokazivač se sastoji iz tri dela, ofseta do početka *tuple-a*, dužine *tuple-a*, a u zavisnosti od verzije može da sadrži i nekoliko kontrolnih bitova koji mogu da utiču na interpretaciju ovog pokazivača. [5] Ovi pokazivači se mogu sresti i pod imenom *line pointers*. [8] Osobina *line pointers-a* je da se oni nikad ne pomeraju, dok se ne oslobode. PostgreSQL svakoj stavci (red u tabeli/tuple ili node u indeksu) dodeljuje jedinstveni *CTID* (*current tuple id*), kao uređeni par (redni broj stranice, indeks *itemId-ja* u okviru stranice).

Konačno, dolazimo i do samih podataka. Redovi u bazi (preciznije *tuples*) se u okviru stranice skladište od pozadi. Svaki *tuple* se sastoji od *header-a* fiksne veličine (23 bajtova na većini mašina),

opcionim *null bitmap*, opciono *OID* polje i sami korisnički podaci. [5] Na slici 6 prikazan je sadržaj *tuple header-a*, a detaljniji opis samih polja biće dati kasnije, obzirom da zahteva poznavanje detaljnije poznavanje *WAL-a* i *MVCC*-ja. Vredi pomenuti i da *tuple* može da se javi u dva oblika – kao „obični“ *tuple* i kao *TOAST tuple*.

| Field       | Type            | Length  | Description                                           |
|-------------|-----------------|---------|-------------------------------------------------------|
| t_xmin      | TransactionId   | 4 bytes | insert XID stamp                                      |
| t_xmax      | TransactionId   | 4 bytes | delete XID stamp                                      |
| t_cid       | CommandId       | 4 bytes | insert and/or delete CID stamp (overlays with t_xvac) |
| t_xvac      | TransactionId   | 4 bytes | XID for VACUUM operation moving a row version         |
| t_ctid      | ItemPointerData | 6 bytes | current TID of this or newer row version              |
| t_infomask2 | uint16          | 2 bytes | number of attributes, plus various flag bits          |
| t_infomask  | uint16          | 2 bytes | various flag bits                                     |
| t_hoff      | uint8           | 1 byte  | offset to user data                                   |

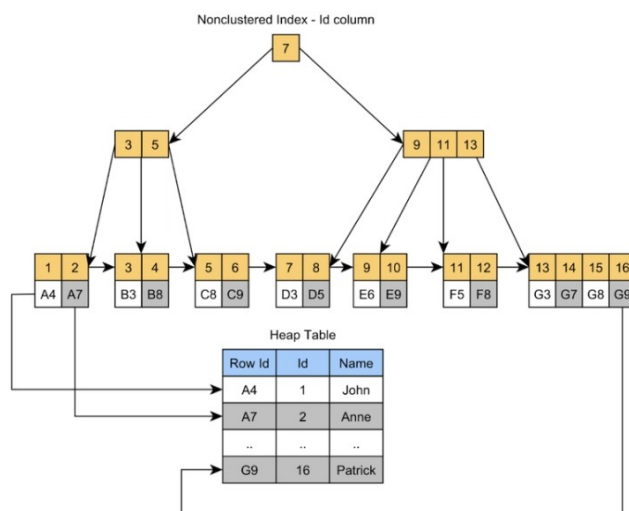
Slika 6 - Layout heap tuple-a, preuzeto iz [5]

Poslednja deo u PostgreSQL stranici je polje *special*. Radi se o slobodnom polju u okviru koga (*table/index*) *access method* može da skladišti proizvoljne podatke. Na primeru Btree indeksa, ovaj deo može da sadrži pokazivač na levi i desni čvor u stablu. [5]

### 3.2 Heap storage

Osnovni i jedini način skladištenja podataka u PostgreSQL-u poznat je pod nazivom *heap storage*. Kao što je ranije pomenuto *table access method* definiše način na koji se podaci smeštaju u okviru stranice. Prilikom instalacije PostgreSQL-a, on dolazi sa tzv. *heap table access method-om*, te odatle i naziv *heap storage*. PostgreSQL, od verzije 12, nudi API u okviru servera za serversko programiranje koji omogućava definisanje custom *table access method-a*. Kao alternativa podrazumevanom načinu skladištenja podataka, postoje i alternativni koji mogu da se instaliraju kao ekstenzija, kao na primer *ZHeap* i *OrioleDB*.

Korišćenje *heap storage-a* podrazumeva da se podaci upisuju u onom redosledu u kom pristižu, nema podrazumevanog automatskog uređivanja (tzv. *clustered indexing*, koji se koristi kod npr. MySQL-a). Svaki indeks, uključujući i indeks nad primarnim ključem je sekundarni. To znači da listovi u indeksu ne sadrže korisničke podatke, već sadrže lokaciju *heap tuple-a* (*CTID*). Na slici 7 prikazan je izgled indeksa nad primarnim ključem kod PostgreSQL-a.



Slika 7 - Indeks nad primarnim ključem kod PostgreSQL-a

### 3.3 MVCC

*MVCC (Multi-Version Concurrency Control)* je strategija za upravljanje konkurentnim pristupom podacima kod relacionih baza podataka. Osnovna ideja je da se u bazi podataka čuva više verzija jednog reda tako da transakcije koje čitaju podatke mogu da pristupe staroj verziji, dok transakcije koje modifikuju red mogu da kreiraju novu. Cilj ovakvog pristupa je da se omogući konkurentni pristup bez zaključavanja *reader-a*. Umesto da se prepíše stara vrednost reda u bazi, baza čuva staru verziju kako bi tekuće transakcije mogle da pristupe verziji koja je bila aktuelna kada je ona počela svoje izvršavanje. [9] Kontrola konkurentnog pristupa nije tema ovog seminarskog rada, ali implikacije koje implementacija *MVCC-a* ima na skladištenje podataka (*insert*, *update*, *delete*) naročito dolaze do izraza kod PostgreSQL-a.

PostgreSQL implementira *MVCC* tako što nove verzije redova skladišti direktno u glavnoj tabeli, a ova tehnika poznata je pod nazivom *tuple versioning*. Zbog takvog pristupa, svaka *update* operacija zapravo prouzrokuje da se u tabelu upíše potpuno nova verzija reda. Ova verzija predstavlja potpunu fizičku kopiju, a ne samo *delta* u odnosu na prethodnu verziju. [9] Kada se kaže da se nova verzija skladišti direktno u glavnoj tabeli, ne misli se da je moguć pristup obema verzijama istovremeno. Staru verziju vide transakcije kod kojih je stara verzija bila aktuelna u trenutku kada su one počele svoje izvršavanje. Kada više ne postoji ni jedna transakcija koja pristupa staroj verziji podatka, on postaje „*dead*“, ali se ne uklanja iz tabele sve dok je *VACUUM* proces ne oslobodi. Slično kao kod ažuriranja, brisanje podataka ne dovodi do fizičkog uklanjanja reda iz baze podataka, već se obeležava *tombstone-om* i oslanja na *VACUUM* proces za oslobađanje. Negativna posledica ovakve implementacije *MVCC-a* je to što svaki *update* zahteva da se izvrši ažuriranje svih indeksa koji postoje nad datom tabelom. Izuzetak kod ovog pravila su *Heap-Only Tuples*.

Sada kada su osnovni detalji načina funkcionisanja *MVCC-a* poznati, moguće je opisati polja koja se nalaze u *header-u heap tuple-a*, prikazanih na slici 6.

- *t\_xmin* – sadrži *transaction id (txid)* transakcije koja je insertovala *tuple*
- *t\_xmax* – sadrži *transaction id (txid)* transakcije koja je izvršila *update* ili brisanje *tuple-a*. Ima vrednost 0 ukoliko je *tuple* najnovija verzija
- *t\_cid* – sadrži broj SQL komandi koje su izvršene u okviru transakcije u kojoj je ovaj *tuple* upisan, koje su izvršene pre komande koja je upisala *tuple*
- *t\_ctid* – sadrži *CTID* aktuelne verzije *tuple-a*. Ukoliko je izvršen *update*, sadrži *CTID* novog *tuple-a*, u suprotnom pokazuje na samog sebe
- *t\_xvac* – polja *t\_cid* i *t\_xvac* su delovi unije u C programskom jeziku, što znači da *header* može da sadrži samo jedno od ovih polja. *t\_xvac* je *legacy* polje i nije u upotrebi u modernim verzijama PostgreSQL-a. [10]
- *t\_infomask2* i *t\_infomask* – kontrolni bitovi koji nose različite informacije o redu. Kroz seminarski rad biće pomenuti neki od kontrolnih bitova.
- *t\_hoff* – ofset do korisničkih podataka

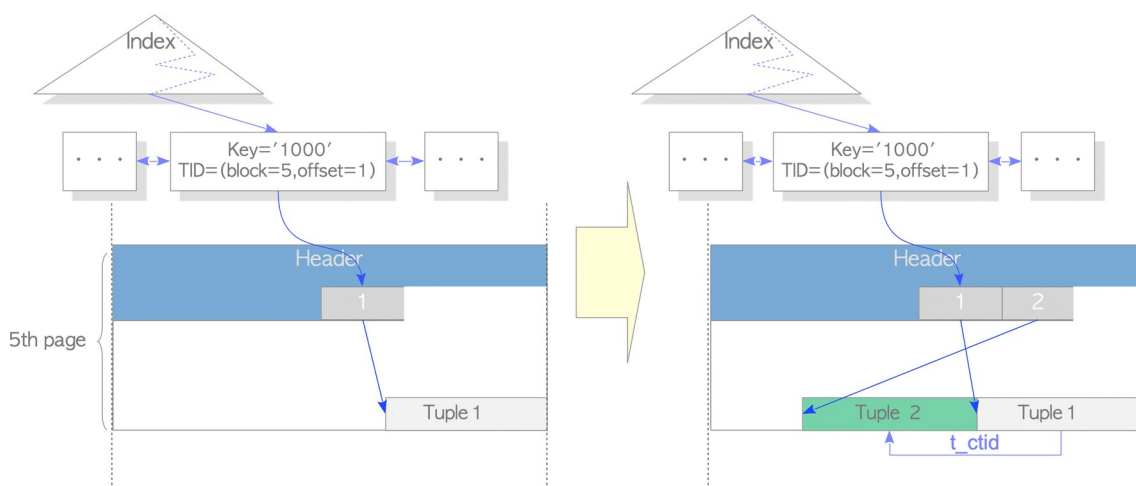
Polje koje sledi *header* kod *tuple-a* je *null bitmap* i koristi se kako bi se naznačile kolone koje imaju null vrednost.

### 3.3.1 Heap-Only Tuples

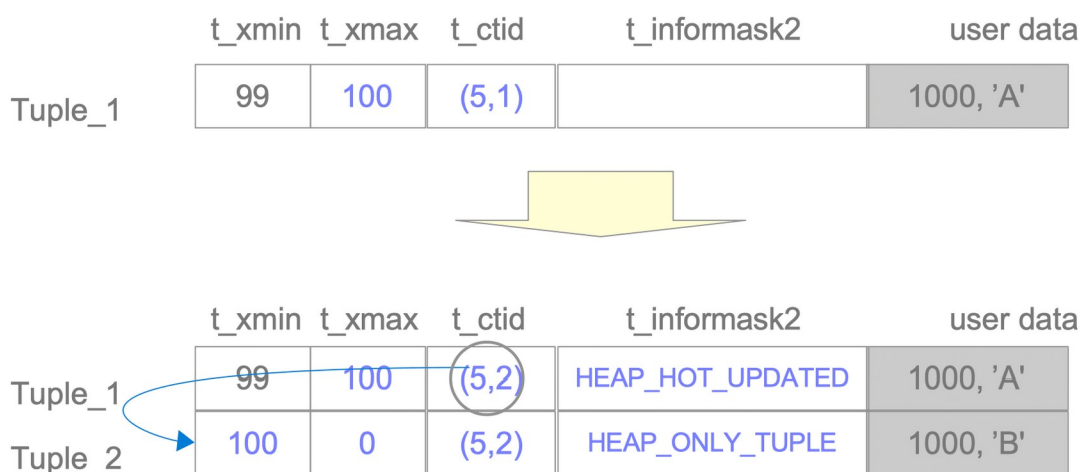
*Heap-Only Tuples* je tehnika koja se koristi kako bi se optimizovala *update* operacija. Kao što je pomenuto *update* operacija dovodi do kreiranja nove fizičke kopije reda. Ova optimizacija za cilj ima da ublaži negativne efekte ovakve implementacije, kada je to moguće. Dva uslova koja moraju biti zadovoljena kako bi bilo moguće primeniti HOT optimizaciju su: [11]

- Update* ne modifikuje bilo koju kolonu nad kojom postoji indeks, ne računajući *summarizing* indekse (*BIRN*).
- Postoji dovoljno slobodnog prostora u stranici koja sadrži stari *tuple* za dodavanje novog *tuple-a*.

*Update* reda korišćenjem HOT optimizacije svakako dovodi do *insert-a* novog *heap tuple-a*, a optimizacija se postiže time što nije neophodno ažurirati sve indekse nad odgovarajućom tabelom. Kada se desi HOT *update* u istu stranicu se insertuje i novi *tuple*, pri čemu se kod starog *tuple-a* u okviru *header* polja *t\_infomask2* postavlja bit *HEAP\_HOT\_UPDATED*, dok se kod novog *tuple-a* postavlja bit *HEAP\_ONLY\_TUPLE*. [12] Opisani postupak prikazan je na slikama 8 i 9.



Slika 8 - Prikaz stranice prilikom *HOT update-a*, preuzeto iz [12]



Slika 9 - Prikaz *tuple header-a* prilikom *HOT update-a*, preuzeto iz [12]

### 3.4 Praktični prikaz

Za potrebe ove praktične demonstracije, kreirana je baza podataka *demo*, kao i jedna tabela u toj bazi. Takođe, instalirana je ekstenzija *pageinspect* koja obezbeđuje funkcije kako bi se stranice baze podataka istražile na nižem nivou.

```
create extension pageinspect;

create database demo;

create table app_user (
    id            int primary key,
    name          varchar(20) not null,
    favorite_animal text
);
```

Slika 10 - Postavke nove baze podataka

Nakon toga je u bazu upisan 1 red - `insert into app_user values (1, 'Luka', 'zebra');`. Sada možemo pogledati kako izgleda stranica koja pripada kreiranoj tabeli.

```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16510
00000000  00 00 00 00 10 99 d2 01 e5 d9 00 00 1c 00 d8 1f |.....|
00000010  00 20 04 20 00 00 00 00 d8 9f 4e 00 00 00 00 00 |. . .....N....|
00000020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00001fd0  00 00 00 00 00 00 00 00 09 03 00 00 00 00 00 00 |.....|
00001fe0  00 00 00 00 00 00 00 00 01 00 03 00 02 08 18 00 |.....|
00001ff0  01 00 00 00 0b 4c 75 6b 61 0d 7a 65 62 72 61 00 |.....Luka.zebra.|
00002000
```

Slika 11 - Izgled stranice table `app_user` nakon inserta 1 reda

Najpre se može primetiti da je veličina stranice 8kB. Prvih 24 bajtova predstavlja *header* stranice, a kako bi se izbeglo njegovor ručno tumačenje, može se iskoristiti funkcija `page_header`, koja dolazi u okviru ekstenzije *pageinspect*.

```
demo=# select * from page_header(get_raw_page('app_user', 0));
   lsn    | checksum | flags | lower | upper | special | pagesize | version | prune_xid
-----+-----+-----+-----+-----+-----+-----+-----+-----
 0/1D29910 |         0 |      0 |    28 | 8152 |    8192 |    8192 |         4 |         0
(1 row)
```

Slika 12 - Rezultat izvršenja funkcije `page_header()`

Zanimljive stavke iz *header-a* su pokazivači *lower* i *upper*. *Lower* pokazivač ukazuje na prvo slobodno mesto sa početka stranice, što je 28. To odgovara veličini *header-a*, i dodatno jedan četvorobajtni pokazivač na podatak, s obzirom da je u tabelu upisan samo jedan red. *Upper* pokazivač ukazuje na slobodno mesto na kraju stranice. Sa slike 11 se može videti da je vrednost jedinog *tuple pointera* `d8 9f 4e 00`. U verziji 18 PostgreSQL-a, nižih 15 bitova se koriste za specifikaciju ofseta, srednja 2 bita se koriste kao flag, a viših 15 bitova određuje dužinu *tuple-a*. Ovim tumačenjem dobijamo da je ofset *tuple-a* 8152, flag ima vrednost 1, a dužina 39 bajtova. Ofset se poklapa sa pokazivačem *upper*, a dužina *tuple-a* se može verifikovati korišćenjem upita sa slike 13.

```
demo=# select pg_column_size(t.*) as row_size_bytes
from app_user as t
where id = 1;
 row_size_bytes
-----
          39
(1 row)
```

Slika 13 - Veličina *tuple-a* u bajtovima

Nad tabelom *app\_user* automatski je kreiran i indeks nad primarnim ključem. Kao što je već pomenuto, ovaj indeks je sekundarni indeks. Na slici 14 prikazan je sadržaj meta stranice indeksa i prve *data* stranice indeksa u hex formatu. Ponovo, kako bi se izbeglo ručno tumačenje ovih vrednosti, ekstenzija *pageinspect* nudi funkciju koja prikazuje sadržaj *data* stranice, koji je prikazan na slici 15.

```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16517
00000000 00 00 00 00 70 99 d2 01 d8 cb 00 00 48 00 f0 1f |....p.....H...|
00000010 f0 1f 04 20 00 00 00 00 62 31 05 00 04 00 00 00 |... ..b1.....|
00000020 01 00 00 00 00 00 00 00 01 00 00 00 00 00 00 00 |.....|
00000030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 f0 bf |.....|
00000040 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00000050 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00001ff0 00 00 00 00 00 00 00 00 00 00 00 00 08 00 00 00 |.....|
00002000 00 00 00 00 b0 99 d2 01 7a 5d 00 00 1c 00 e0 1f |.....z].....|
00002010 f0 1f 04 20 00 00 00 00 e0 9f 20 00 00 00 00 00 |... ..|
00002020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00003fe0 00 00 00 00 01 00 10 00 01 00 00 00 00 00 00 00 |.....|
00003ff0 00 00 00 00 00 00 00 00 00 00 00 00 03 00 00 00 |.....|
00004000
```

Slika 14 - Sadržaj indeksa u hex formatu

```
demo=# select * from bt_page_items('app_user_pkey', 1);
 itemoffset | ctid  | itemlen | nulls | vars | data | dead | htid | tids
-----+-----+-----+-----+-----+-----+-----+-----+-----
          1 | (0,1) |      16 | f      | f      | 01 00 00 00 00 00 00 00 | f      | (0,1) |
(1 row)
```

Slika 15 - Sadržaj indeksa korišćenjem *bt\_page\_items()* funkcije

Sa slike 15 se može videti da je sadržaj jedinog lista u indeksu pokazivač (0,1), koji ukazuje da se vrednost podatka nalazi na stranici (bloku) 0, a radi se o prvom redu na toj stranici.

Sada možemo ažurirati vrednost postojećeg reda, korišćenjem narednog upita: `update app_user set favorite_animanl = 'lion' where id = 1;` Sada je na slici 16 prikazan sadržaj stranice tabele u hex formatu, na slici 17 je prikazan sadržaj *header-a* stranice, na slici 18 sadržaj indeksa, a na slici 19 sadržaj informacionih bitova iz *header-a* podataka na stranici.

Sa slike 16 može da se primeti očekivano *heap storage* ponašanje, gde je novi red upisan u tabelu, ali je stari red takođe još uvek tu. Sa slike 17 se može uočiti da je granica *upper* pomerena za veličinu jednog *tuple-a*. Slika 18, međutim prikazuje da se u indeksu nalazi samo jedan ulaz koji i dalje ukazuje na stari red. Ovo je takođe očekivano ponašanje, obzirom da su ispunjeni uslovi za primenu HOT optimizacije. Ovo potvrđuje i slika 19 gde vidimo flagove koji se odnose na nju.



```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16510
00000000  00 00 00 00 80 d4 d5 01 4e b8 00 00 20 00 b0 1f |.....N... ..|
00000010  00 20 04 20 0a 03 00 00 d8 9f 4e 00 b0 9f 4e 00 |. . ....N...N.|
00000020  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00001fb0  0a 03 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
00001fc0  02 00 03 80 02 29 18 00 01 00 00 00 0b 4c 75 6b |.....).....Luk|
00001fd0  61 0d 70 61 6e 64 61 00 09 03 00 00 0a 03 00 00 |a.panda.....|
00001fe0  00 00 00 00 00 00 00 00 02 00 03 40 02 05 18 00 |.....@....|
00001ff0  01 00 00 00 0b 4c 75 6b 61 0d 7a 65 62 72 61 00 |.....Luka.zebra.|
00002000
```

Slika 16 - Sadržaj stranice tabele nakon ažuriranja u hex formatu

```
demo=# select * from page_header(get_raw_page('app_user', 0));
   lsn    | checksum | flags | lower | upper | special | pagesize | version | prune_xid
-----+-----+-----+-----+-----+-----+-----+-----+-----
 0/1D5D480 |    26147 |    0  |    32 |   8112 |    8192 |    8192 |        4 |         778
(1 row)
```

Slika 17 - Sadržaj header-a stranice nakon ažuriranja reda

```
demo=# select * from bt_page_items('app_user_pkey', 1);
 itemoffset | ctid  | itemlen | nulls | vars |          data          | dead | htid  | tids
-----+-----+-----+-----+-----+-----+-----+-----+-----
          1 | (0,1) |      16 | f     | f    | 01 00 00 00 00 00 00 00 | f    | (0,1) |
(1 row)
```

Slika 18 - Sadržaj data stranice indeksa nakon ažuriranja

```
demo=# SELECT t_ctid, raw_flags, combined_flags
        FROM heap_page_items(get_raw_page('app_user', 0)),
        LATERAL heap_tuple_infomask_flags(t_infomask, t_infomask2)
        WHERE t_infomask IS NOT NULL OR t_infomask2 IS NOT NULL;
 t_ctid | raw_flags | combined_flags
-----+-----+-----
 (0,2)  | {HEAP_HASVARWIDTH,HEAP_XMIN_COMMITTED,HEAP_XMAX_COMMITTED,HEAP_HOT_UPDATED} | {}
 (0,2)  | {HEAP_HASVARWIDTH,HEAP_XMIN_COMMITTED,HEAP_XMAX_INVALID,HEAP_UPDATED,HEAP_ONLY_TUPLE} | {}
(2 rows)
```

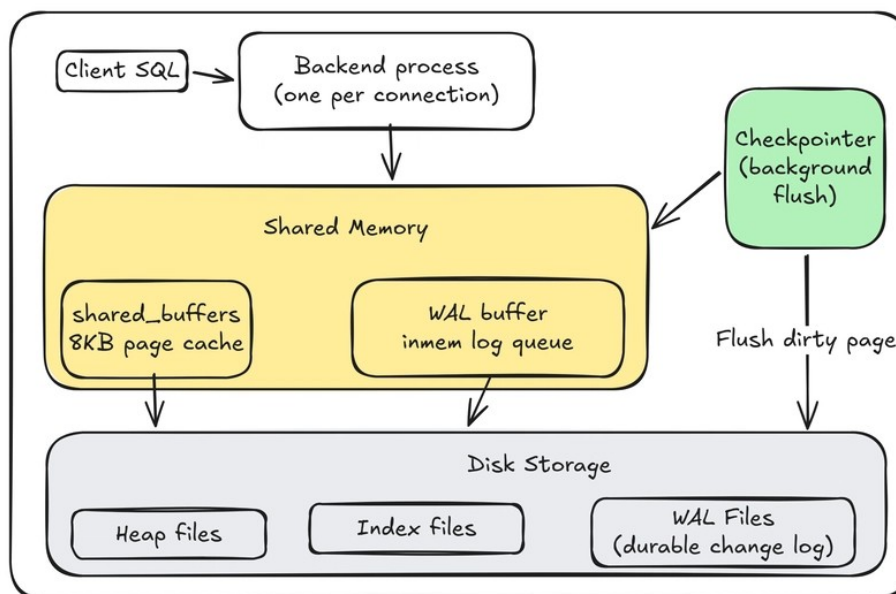
Slika 19 - Sadržaj headera tuple-ova u stranici tabele nakon ažuriranja

## 4 PostgreSQL write path i read path

U prethodnom poglavlju predstavljene su osnove skladištenja podataka u PostgreSQL-u. Fokus je bio stavljen na osnove kao što su stranica, *heap* i *tuple*. Drugim rečima, prikazan je pogled na strukturu organizacije podataka kod PostgreSQL-a, bez uzimanja u obzir interakcije sa memorijom i diskom. *Table access method* i *manager* nude određeni nivo apstrakcije kroz interfejs koji sakriva složenu interakciju sa memorijom i diskom. Na početku poglavlja, na slici 3 prikazana je arhitektura PostgreSQL baze podataka. Kao što se sa slike može videti veći deo *storage* mehanizma kao, što ujedno predstavlja i mehanizam koji omogućava prethodno pomenuti nivo apstrakcije zauzima interakcija sa baferima i diskom, kroz posredstvo operativnog sistema. U ovom poglavlju biće prikazan put koji podaci prolaze kada se izvrši modifikacija ili čitanje podataka, bez zalaženja u detalje pojedinačnih sistema. Naredna poglavlja biće posvećena detaljnijem opisu glavnih podsistema koji se prolaze prilikom čitanja i upisa podataka u bazu.

### 4.1 Arhitektura

Na slici 20 prikazane su četiri glavne komponente koje učestvuju u bilo kojoj operaciji nad bazom podataka.



Slika 20 - Komponente koje učestvuju u svi operacijama, preuzeto iz [14]

Prilikom podizanja PostgreSQL baze podataka, kreira se tzv. *postmaster* proces. Ovaj proces podrazumevano osluškuje dolazeće konekcije na portu 5432 i za svaku korisničku konekciju kreira novi (backend) proces. Pored ovih procesa, postoji veliki broj *background worker-a* koji upravljaju različitim aspektima PostgreSQL-a, od *VACUUM-a* do *flush-ovanja* prljavih stranica. Stavka koja je izostala sa slike 20 je lokalna memorija koja se alocira i dodeljuje svakom forkovanom backend procesu i memorija koja se alocira za različite *background worker-e*. [13]

#### 4.1.1 Lokalna memorija

Lokalna memorija backend procesa sastoji se od nekoliko delova koji imaju različite namene:

- Radna memorija – deo memorije koji je rezervisan za sortiranje i *hash* operacije. SQL klauzule kao što su *ORDER BY*, *DISTINCT*, *merge joins*, *hash joins*, *hash-based* agregacije i



procesiranje *IN* podupita oslanjaju se na ovaj deo memorije. Veličina ovog dela memorije može se kontrolisati korišćenjem *work\_mem* parametra u PostgreSQL konfiguraciji. Takođe treba naglasiti da u slučaju da upit sadrži veći broj pomenutih operacija, za svaki korak se alokira *chunk* memorije veličine *work\_mem*, tako da se pod pojmom radna memorija zapravo podrazumeva kolekcija ovih *chunk*-ova.

- *Maintenance* radna memorija – ovaj deo memorije koristi se za operacije kao što su *VACUUM*, *CREATE INDEX*, *ALTER TABLE ADD FOREIGN KEY* i sl. Veličina ovog dela memorije kontroliše se parametrom *maintenance\_work\_mem*. U jednom trenutku jedna klijentska sesija može da izvršava samo jednu *maintenance* operaciju, tako da se *maintenance\_work\_mem* može postaviti na nešto veću vrednosti.
- *Autovacuum* radna memorija – ovaj deo lokalne memorije definiše maksimalnu količinu memorije koju će svaki *autovacuum worker* koristiti. Kontroliše se korišćenjem parametra *autovacuum\_work\_mem*. Podrazumevano ova vrednost postavljena je na -1, što označava da se koristi parametar *maintenance\_work\_mem*. Maksimalna vrednost koja se može dodeliti jednom *autovacuum worker*-u je 1GB.
- *Temporary buffers* – PostgreSQL podržava pomoćne (privremene) tabele za koje je takođe potrebno alocirati neki prostor. Ovaj deo memorije ima tu ulogu. Veličina se kontroliše parametrom *temp\_buffers*. [15]

#### 4.1.2 Shared memory

Pored lokalne memorije koja se alokira za svaki proces, PostgreSQL server alokira i deljivu memoriju u RAM-u prilikom pokretanja. Ova memorija je zajednička za sve PostgreSQL instance, a ne za pojedinačne baze unutar klastera. PostgreSQL alokira veoma malu količinu *System V deljive memorije* i mnogo veću količinu anonimne *mmap* deljive memorije. Postgre takođe koristi po jedan System V semafor za svaki proces (korisnički ili *background worker*). Slično kao i lokalna memorija, deljiva memorija je podeljena na delove.

- *Shared Buffers* – deo deljive memorije koju je alokirao PostgreSQL kome se pristupa radi upisa i čitanja podataka. Prilikom pribavljanja podataka, prvo se proverava da li su dostupni u *shared buffers* delu, a zatim se ide do diska. Isto važi i za DML operacije. Podaci se prvo upisuju u *shared buffers*, a nakon toga se promene *flush-uju* na disk. Izuzetak od ovog pravila su privremene (*temporary*) tabele, koje se, kao što je već pomenuto, smeštaju u *temp buffers* delu lokalne memorije. Veličina ovog dela memorije je fiksna, alokira se prilikom pokretanja baze i ne menja se tokom života. Veličina *shared buffers* dela kontroliše se korišćenjem parametra *shared\_buffers*. Preporuka je da na serveru sa više od 1GB RAM memorije, veličina bude oko 25% sistemske memorije.
- *WAL Buffers* – ovaj deo memorije se koristi kako bi se *WAL record*-i privremeno čuvali pre nego što se upišu na disk. *WAL writer* proces koji je zadužen da stavke iz ovog bafera upisuje na disk. Veličina ovog dela memorije kontroliše se *wal\_buffers* parametrom i preporuka je da bude oko 3% veličine deljive memorije. Osnovna uloga ovog dela je *batching wal recorda* koje treba presliakti na disk. Precizniji opis benefita koji se dobijaju ovim biće opisan kasnije, prilikom razmatranja *write path-a*.
- *CLOG Buffers* – PostgreSQL privremeno čuva statuse transakcija u ovom delju deljive memorije pre nego što budu upisani na disk. *DBMS* automatski kontroliše veličinu ovog

segmenta. Prilikom gašenja servera sadržaj ovog bafera se upisuje u `pg_xact` poddirektorijum.

- *Lock Space* – deo memorije koji je namenjen za upravljanje *lock-ovima*. PostgreSQL podržava nekoliko različitih tipova *lock-ova* (*row-level*, *table-level* i *transaction-level*). Slično prethodnom delu deljive memorije, DBMS automatski upravlja i veličinom ovog segmenta. [15]

#### 4.1.3 Disk storage

U poglavlju 2 ovog seminarskog rada prikazan je fizički *layout* poddirektorijuma u okviru kojih se skladište fajlovi. U poglavlju 3 prikazan je izgled *heap* (ujedno i *index*) fajlova. Prema tome fajlovi koji pripadaju *disk storage-u* a još ih treba pomenuti su:

- *WAL* fajlovi – predstavljaju sekvencijalni zapis svih promena koje su se desile u bazi podataka. *WAL* fajlovi obezbeđuju durabilnost tako što omogućavaju da sve promene koje su se desile nad bazom podataka mogu da budu ponovljene u slučaju pada baze podataka.
- *WAL* arhivski fajlovi – PostgreSQL nudi mogućnost da se *WAL* fajlovi arhiviraju van direktorijuma baze podataka i na ovaj način obezbeđuju dodatni nivo sigurnosti.
- Konfiguracioni fajlovi – tekstualni fajlovi koji se koriste za podešavanje servera baze podataka. Osnovni konfiguracioni fajlovi su *postgresql.conf* (serverska podešavanja), *postgresql.auto.conf* (automatska serverska podešavanja), *pg\_hba.conf* (klijentska autentifikacija), *pg\_ident.conf* (mapiranja korisnika).
- *Log* fajlovi – kolekcija događaja i poruka koji se generišu u toku operativnog ciklusa baze podataka. Oni se koriste za dijagnostiku, analizu aktivnosti i performansi baze podataka.

## 4.2 Write path

Pod „*write path*“ podrazumeva se opis delova sistema i redosled kojim oni izvršavaju akcije kada se izvršava neka *DML* operacija. Kao kod bilo kog upita, prvi korak je parsiranje SQL naredbe u AST, formiranje jednostavnog plana izvršenja. Kod *DML-a* nema kompleksnog planiranja već se prvobitni, jednostavan plan odmah predaje *executor-u* na izvršavanje. [14]

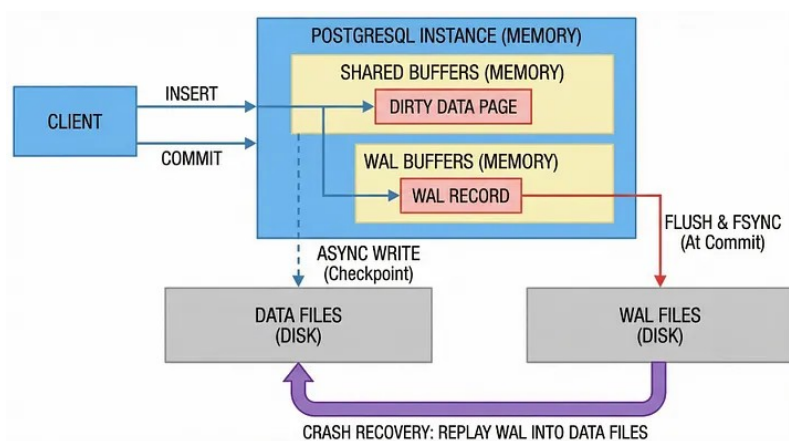
Kao što je već pomenuto, PostgreSQL skladišti korisničke podatke iz tabela u fiksnim stranicama veličine 8kB. Ove stranice se perzistentno čuvaju na disku. Pre izvršenja operacije, određuju se stranice koje će biti modifikovane. Ako se stranice već nalaze u *shared buffers*, Postgre će iskoristiti ove kopije iz memorije. Ukoliko stranica nije u *shared buffers*, Postgre zahteva čitanje stranice od operativnog sistema. Operativni sistem stranicu može dovući iz svog keša, a ukoliko se ne nalazi u tom kešu, tek tada se odlazi i zahteva čitanje sa diska.

Nakon što je obezbeđeno da je odgovarajuća stranica u memoriji, vrši se njena modifikacija. Ova stranica sada postaje *dirty page*. Međutim, pre nego što Postgre potvrdi uspešno izvršenu modifikaciju, potrebno je da obezbedi *durability*. Za svaku promenu koja se desi nad stranicom, generiše se *WAL record* koji opisuje tu promenu. *WAL record* se inicijalno upisuje u *WAL buffers*, a nakon toga se upisuje i u *WAL* fajlove na disku.

Potrebno je takođe obratiti pažnju i na indekse. Nakon što je podatak upisan u odgovarajuću stranicu u *shared buffers*, potrebno je ažurirati i sve indekse koji postoje nad odgovarajućom

tabelom (zanemarujemo mogućnost da se desila HOT optimizacija). To podrazumeva pronalaženje lista u stablu indeksa gde se nalazi odgovarajući pokazivač i upis novog podatka. Moguće je da dođe i do splitovanja stranice. Ažuriranje indeksa prolazi kroz isti *write path* kao i modifikacija tabele, što znači modifikaciju stranice u *shared buffers*, dovlačenje stranice sa diska ukoliko ne postoji, eventualna sinhronizacija, kao i beleženje u WAL.

Podaci koji su upisani u stranicu koja se nalazi u *shared buffers* se asinhrono, korišćenjem *background worker-a* upisuje na disk. Ukoliko dođe do pada baze podataka nakon što su podaci upisani na disk, oni će preživeti. Isto, ukoliko su podaci upisani u WAL fajlove na disku, u slučaju pada baze podataka, promene koje su se desile nad bazom će biti ponovo izvršene zahvaljujući zapisima iz WAL-a. Na slici 21 vizuelno je prikazan *write path*. [16]



Slika 21 - Vizuelni prikaz *write path-a*

#### 4.2.1 Potvrda commit-a

U prethodnom tekstu izostavljeno je specificiranje trenutka u kome PostgreSQL potvrđuje, odnosno obaveštava klijenta da je operacija *commit-ovana*. PostgreSQL je fleksibilan po ovom pitanju i nudi nekoliko mogućnosti:

- Izmena je uspešno zabeležena u WAL – ovo je podrazumevana i najbezbednija mogućnost. Vredi pomenuti da kada se izda naredba za upis podatka na disk, operativni sistem tu izmenu može zabeležiti u svom kešu, a ne zapravo na disk. Disk je mnogo sporiji od memorije tako da se često modifikacije gomilaju, a nakon toga sve zajedno *flush-uju*. Postoji sistemski poziv *fsync* koji blokira pozivajući proces sve dok operativni sistem ne da garanciju da je podatak upisan na disk. Ova mogućnost koristi pomenuti sistemski poziv. Kada se izda zahtev za upis u WAL koristi se *fsync*, odgovarajući proces se blokira dok se ne potvrdi upis na disk. Kada je podataka bezbedno upisan na disk, PostgreSQL potvrđuje *commit*. *Fsync* sistemski poziv, pored upisivanja korisničkih podataka na disk vrši ažuriranje svih metapodataka koji se odnose na fajl.
- Fdatasync* – druga mogućnost koju PostgreSQL nudi se od prve razlikuje samo po sistemskom pozivu koji se koristi. U ovom slučaju koristi se *fdatasync* sistemski poziv. Razlika između ovog i prethodnog sistemskog poziva je u tome što kod *fdatasync-a* operativni sistem vraća potvrdu da je fajl ažuriran odmah nakon što je korisnički podatak upisan na disk. Ne čeka se ažuriranje metapodataka. U principu, *fdatasync* sistemski poziv je dovoljan kod PostgreSQL-a zbog toga što se za vraćanje baze u ispravno stanje koriste samo podaci iz fajla, a ne metapodaci. Nivo bezbednosti koji *fsync* obezbeđuje, a *fdatasync* ne obezbeđuje odnosi se na greške koje mogu biti prouzrokovane zbog operativnog sistema.

- *async\_commit* – PostgreSQL ima mogućnost i da ne čeka. Promena se upisuje u *WAL buffers* u memoriji nakon čega se izdaje potvrda da je *commit* uspešan.

Kompromis između ovih mogućnosti je jasan. Korišćenje *fsync* sistemskog poziva garantuje najviši nivo *durability*-ja, po dosta visoku cenu propusnosti. Sa druge strane korišćenje asinhronog *commit*-a daje najslabije *durability* garancije, ali se propusnost značajno povećava. [16]

#### 4.2.2 Optimizacija potvrde commita-a

*Fsync* sistemski poziv je izuzetno skup. Iz tog razloga PostgreSQL ima nekoliko mehanizama koji mogu da omoguće da se ublaži ova cena. Neke optimizacije se primenjuju na nivou same baze, a neke se primenjuju na aplikacionom nivou.

##### 4.2.2.1 Grupni commit

Kada veći broj transakcija komituje u isto vreme, Postgre može da ih grupiše zajedno i da njihove *WAL* *recorde* upiše na disk kroz jedan sistemski poziv. Umesto da svaka transakcija trigeruje novi poziv i upis na disk, izdaje se samo jedan zahtev za upis. Ova optimizacija se dešava automatski i korisnik nema kontrolu nad time da li će se ona desiti ili ne. Međutim, grupni *commit* u mnogome zavisi od tajminga. Transakcije se mogu grupisati samo ako su njihova *commit* vremena dovoljno blizu. U suprotnom, nije moguće grupisati ih. Iz tog razloga nešto veći konkurentni pristup može značajno povećati propusnost baze podataka, obzirom da se na taj način povećava verovatnoća da će transakcije biti grupisane. [16]

##### 4.2.2.2 Batch transakcije

Ova optimizacija se ne vrši na nivou baze, već se očekuje njena implementacija na aplikacionom nivou. Ona podrazumeva da se veći broj *DML* naredbi grupiše i izvrši u okviru jedne transakcije. Motiv je prilično jasan, ukoliko se nakon svake *DML* naredbe očekuje *fsync* poziv to značajno može da usporavi bazu. Sa druge strane prilikom grupisanja operacija u *batch*, veći broj stranica u *shared buffers* će biti modifikovan, generisaće se *WAL record* za svaku promenu, ali će svi ti *WAL* zapisi biti upisani na disk jednim pozivom *fsync*-a.

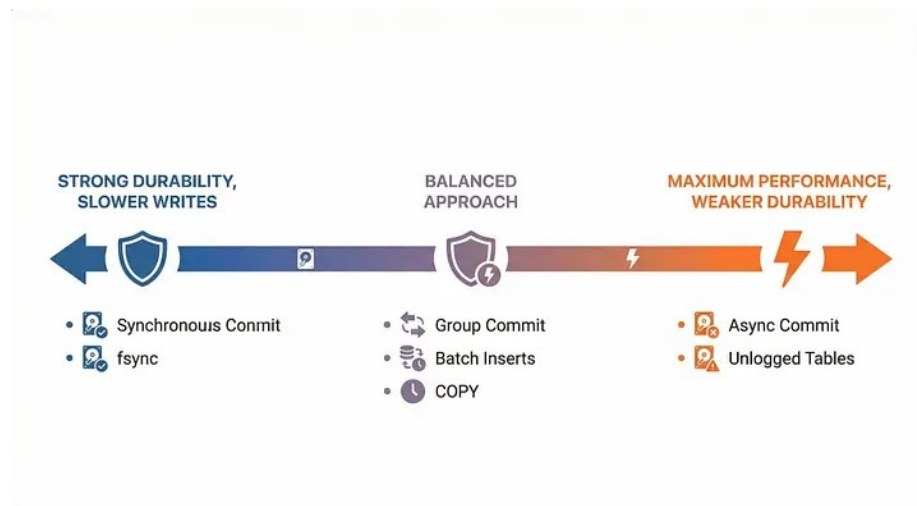
##### 4.2.2.3 COPY

*COPY* naredba predstavlja najefikasniji način za *insert* ogromne količine podataka direktno u PostgreSQL. *COPY* naredba učitava sve redove u jednu komandu umesto izvršavanja velikog broja *INSERT* naredbi. Prednosti koje se dobijaju je i u tome što se naredba parsira i planira samo jednom, za razliku od *batch insert*-a. Dodatno, *WAL* zapisi koji se generišu su nešto efikasniji, obzirom da mogu da se generišu kao jedan veći *chunk*, umesto većeg broja manjih zapisa. *COPY* naredba uklanja veći deo nepoželjnog *overhead*-a koji postoji prilikom korišćenja *batch insert*-a.

##### 4.2.2.4 Unlogged tables

Sve pomenute optimizacije se odnose na optimizaciju *WAL*-a i efikasnosti sistemskog poziva. Ukoliko *WAL* ne postoji, nema potrebe za bilo kojom vrstom optimizacije. Unlogged tables predstavljaju upravo to – tabele čije promene se ne reflektuju u *WAL*-u. Kada se modifikuje neki podatak u *unlogged* tabeli, promena se upisuje u odgovarajuću stranicu u memoriji i eventualno *flush*-uje na disk, ali se ne generišu *WAL* zapisi. Zbog toga *DML* operacije nad ovakvima tabelama su značajno brže od klasičnih tabela. Međutim, kompromis je jasan. U slučaju da dođe do pada baze

podataka, pre nego što su svi podaci upisani na disk, oni će biti izgubljeni. [16] Na slici 22 prikazan je odnos pomenutih metoda u pogledu performansi i *durability* garancije.



Slika 22 - Grafički prikaz odnosa različitih *commit* mogućnosti, preuzeto iz [16]

### 4.3 Read path

*Read path* je nešto jednostavniji od *write path-a*. Kompleksnost koja postoji kod *read path-a*, a koja ne postoji kod *write path-a* je planiranje koje se vrši kao korak nakon parsiranja upita.

Ukoliko se pretraga vrši po koloni koja je indeksirana (i ukoliko PostgreSQL proceni da treba iskoristiti indeks) krenuće se u *index-scan*. Indeksi su, kao i tabele, podeljeni u stranice veličine 8kB. Prilikom pristupa stranici indeksa, prvo se proverava da li se ona nalazi u *shared buffers*. Većina stranica koje sadrže interne čvorove stabla se veoma često nalaze u *shared buffers*. Ukoliko se stranica ne nalazi u *shared buffers*, ona se pribavlja sa diska (ili iz keša operativnog sistema, ukoliko je keširana tamo). [14]

Kao što je već pomenuto svi indeksi kod PostgreSQL-a su sekundarni. To znači da ulaz u listu indeksa sadrži *CTID* podatka, koji se sastoji od identifikatora stranice i indeksa podatak u okviru stranice. Nakon što se pronađe *CTID*, pribavlja se odgovarajuća stranica i nalazi se podatak unutar nje.

Ukoliko se pretraga vrši po koloni koja nije indeksirana (ili jeste indeksirana, ali je PostgreSQL procenio da ne treba iskoristiti indeks), tada se obilaze sve stranice koje pripadaju toj tabeli i proverava se da li se podataka nalazi unutar date stranice. Pribavljanje stranica i ovde može da označava direktan pristup stranici, ukoliko se ona nalazi u *shared buffers* ili njeno dovlačenje sa diska u *shared buffers*.

Donošenje odluke o tome da li će se koristiti indeks prilikom pretraživanja podataka zavisi od procenjene selektivnosti. Ukoliko je selektivnost mala, sekvencijalna pretraga svih stranica može da bude brža od pretrage indeksa zbog načina pristupa memoriji. Kod sekvencijalnog pristupa, maksimalno se iskorišćava arhitektura fizičke memorije i operativnog sistema. Upotreba indeksa u ovoj situaciji može da pomoći u pronalaženju stranica koje treba dovući, ali te stranice mogu da budu rasute po memoriji, što usporava pristup. PostgreSQL koristi i tehniku optimizacije koja se zove *bitmap index scan* za ovakve slučajeve – pribavljaju se svi *CTID*-ovi, sortiraju se po stranici kojoj pripadaju, što može random pristup stranicama da približi sekvencijalnom. [14]

## 4.4 Praktični prikaz

U okviru praktičnog prikaza ovog poglavlja biće prikazano kako izgleda *write path*, poređenje optimizacija opisanih u delu 4.2.2. kao i izgled *read path-a*. Biće iskorišćena i ekstenzija *pg\_buffercache*, koja obezbeđuje uvid u metapodatke o *shared buffers*.

### 4.4.1 Write path

Za potrebe ovog dela biće korišćena tabela koja je prikazana na slici 10, koja je inicijalno bez ikakvih podataka. Pre svega možemo da izvršimo upit kojim se proverava broj stranica u *shared buffers*. Rezultat izvršenja ovog upita prikazan je na slici 23.

```
demo=# select count(*) from pg_buffercache;
count
-----
 16384
(1 row)
```

Slika 23 - Rezultat izvršenja upita kojim se prikazuje broj ulaza u *shared buffers*

Dobijeni rezultat je 16384. Baza podataka sadrži samo 1 korisničku tabelu u koji ništa nije upisano. To znači da se prilikom podizanja baze u *shared buffers* kešira više od 16 hiljada stranica. Međutim, detaljnijom analizom može da se utvrdi da je samo nešto više od 1000 slotova u upotrebi, kao što pokazuje upit na slici 24. To znači da se prilikom podizanja baze u *shared buffers* kreira onoliko slotova koliko *shared buffers* može da prihvati. Podrazumevana veličina je 128MB, veličina 1 slota je 8KB, a deljenjem ova dva dobijamo broj 16384.

```
demo=# select reldatabase, count(*) from pg_buffercache group by reldatabase;
reldatabase | count
-----+-----
           | 15145
          5 |   237
           0 |    59
        16454 |   844
           1 |    99
(5 rows)
```

Slika 24 - Raspodela slotova u *shared buffers* po bazama

```
demo=# select bufferid, isdirty, relname
from pg_buffercache b join pg_class c
      on b.relfilenode = c.relfilenode
where b.reldatabase = 16454 and b.isdirty = true;
bufferid | isdirty | relname
-----+-----+-----
(0 rows)
```

Slika 25 - Broj slotova u *shared buffers* koji pripadaju tabeli *app\_user* ili indeksu nad njom

Sa slike 25 se može videti da inicijalno u *shared buffers* ne postoji ni jedan slot u kome se nalazi stranica koja pripada tabeli *app\_user* ili indeksu nad tom tabelom. Sada možemo da upišemo jedan red u tabelu, a rezultat izvršenja tog upita prikazan je na slici 26.

```
demo=# explain (analyze, buffers) insert into app_user values (1, 'Luka', 'zebra');
               QUERY PLAN
-----
Insert on app_user  (cost=0.00..0.01 rows=0 width=0) (actual time=0.305..0.305 rows=0.00 loops=1)
  Buffers: shared hit=14 read=1 dirtied=3 written=2
    -> Result  (cost=0.00..0.01 rows=1 width=94) (actual time=0.002..0.003 rows=1.00 loops=1)
Planning Time: 0.046 ms
Execution Time: 0.326 ms
(5 rows)
```

Slika 26 - Upis jednog reda u tabelu *app\_user*

Ukoliko se odmah nakon izvršenja upita sa slike 26 prikaže sadržaj fajla na disku, može se primetiti da je fajl prazan, što je prikazano na slici 27. Pored toga, upit sa slike 28 prikazuje status slotova u *shared buffers* koji su dodeljeni tabeli *app\_user* i indeksu nad tom tabelom.

```
lule@lule-Asus-Vivobook:~$ sudo hexdump -C /var/lib/postgresql/18/main/base/16454/16588
00000000 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
*
00002000
```

Slika 27 - Sadržaj fajla na disku neposredno nakon izvršenja upita sa slike 26

```
demo=# select bufferid, isdirty, relname
from pg_buffercache b join pg_class c
    on b.relfilenode = c.relfilenode
where c.relname like '%app_user%' and b.isdirty = true;
 bufferid | isdirty |   relname
-----+-----+-----
       729 | t       | app_user_pkey
       730 | t       | app_user_pkey
       731 | t       | app_user
(3 rows)
```

Slika 29 - Slotovi dodeljeni objektima nad tabelom *app\_user* nakon izvršenja upita sa slike 26

Jasno se može uočiti da je promena zabeležena u slotovima u *shared buffers*, ali još uvek nije sinhronizovana sa diskom. Takođe, stranice u slotovima imaju postavljen flag *dirty*. Izvršenje ovog upita nakon što se promene *flush-uju* dovodi do situacije koja je prikazana na slici 25. Dodatno, možemo potvrditi da je promena zabeležena u WAL, čime je obezbeđen *durability* i bez istovremenog upisivanja na disk.

```
lule@lule-Asus-Vivobook:~$ sudo /usr/lib/postgresql/18/bin/pg_waldump /var/lib/postgresql/18/main/pg_wal/0000000010000000000000000001 --relation=1663/16454/16588
rmgr: Heap          len (rec/tot):    70/    70, tx:          837, lsn: 0/01FB1F30,
prev 0/01FB1EF8, desc: INSERT+INIT off: 1, flags: 0x00, blkref #0: rel 1663/164
54/16588 blk 0
```

Slika 28 - Prikaz reda iz WAL-a nakon upisa reda



Sa slike 28 se može videti da su iskorišćena 3 slota u *shared buffers*. To odgovara očekivanju, obzirom da je ažurirana jedna stranica na disku, kao i dve stranice indeksa – stranica sa metapodacima i stranica sa podacima.

Konačno, možemo analizirati rezultat izvršenja upita sa slike 26. Upis u tabelu izvršen je u sklopu klauzule *explain (analyze, buffers)* čime je prikazan plan izvršenja upita i pristup slotovima u *shared buffers*. Kao što se sa slike može primetiti, PostgreSQL je pristupio većem broju slotova. Kao posledica upisa 3 slotova su dobila oznaku *dirty*. Naredni upis u bazu prouzrokuje značajno manji broj pristupa slotovima, kao što je prikazano na slici 30.

```
demo=# explain (analyze, buffers) insert into app_user values (2, 'Pera', 'panda');
                                QUERY PLAN
-----
Insert on app_user (cost=0.00..0.01 rows=0 width=0) (actual time=0.048..0.049 rows=0.00 loops=1)
  Buffers: shared hit=2 dirtied=1
-> Result (cost=0.00..0.01 rows=1 width=94) (actual time=0.002..0.002 rows=1.00 loops=1)
Planning Time: 0.067 ms
Execution Time: 0.070 ms
(5 rows)
```

Slika 30 - Upis novog reda u tabelu app\_user

Kao poslednju proveru, na slici 31 je prikazan rezultat upisa većeg broja podataka. Rezultat se ne razlikuje od prethodnih upisa, osim što je iskorišćen veći broj slotova.

```
demo=# explain (analyze, buffers)
insert into app_user (id, name, favorite_animal)
select
  gs,
  'Luka' || gs,
  'zebra' || gs
from generate_series(3, 10003) as gs;
                                QUERY PLAN
-----
Insert on app_user (cost=0.00..275.03 rows=0 width=0) (actual time=24.810..24.811 rows=0.00 loops=1)
  Buffers: shared hit=29765 dirtied=91 written=94
-> Function Scan on generate_series gs (cost=0.00..275.03 rows=10001 width=94) (actual time=1.254..4.146 rows=10001.00 loops=1)
Planning Time: 0.137 ms
Execution Time: 24.846 ms
(5 rows)
```

Slika 31 - Upis 10000 podataka u tabelu app\_user

## 4.4.2 Optimizacija potvrde commit-a

Ovaj deo praktičnog prikaza odnosi se na optimizacije koje mogu da se izvrše kada se koristi sinhroni komit korišćenjem *fsync* sisemskog poziva. U te optimizacije spadaju: grupni commit, korišćenje batch insert naredbi, korišćenje COPY komande.

### 4.4.2.1 Grupni commit

Najpre ćemo testirati grupni *commit*. Za potrebe testiranja grupnog commit-a kreirana je posebna baza podataka korišćenjem naredbe `create database gc_test;`. Nakon toga unutar te baze kreirana je jednostavna tabela korišćenjem naredbe `create table t_gc (id bigserial primary key, v text);`

Cilj testa je da se pokaže da se korišćenjem *grupnog commit-a* dobija veća propusnost. Od značaja su podešavanja PostgreSQL-a *synchronous\_commit*, *commit\_delay* i *commit\_siblings*. *Synchronous\_commit* kontroliše da li će PostgreSQL čekati potvrdu od operativnog sistema da se podatak upisan u WAL pre nego što pošalje potvrdu korisniku. Za potrebe ovog testa postavljena je vrednost „ON“. *Commit\_delay* predstavlja kašnjenje pre pokretanja WAL flush-a, u mikrosekundama. Njegova vrednost tokom testa je 0. Konačno, *commit\_siblings* predstavlja minimalni broj aktivnih transakcija da bi se *commit\_delay* vreme čekalo. Njegova vrednost je 5.



Za potrebe ovog testa korišćen je *pgbench*. Radi se o jednostavnom *command-line* alatu koji dolazi uz PostgreSQL i koristi se za merenje performansi, kao npr. broj transakcija u sekundi (*TPS*). Kreirana je jednostavna skripta koja će se koristiti za ovaj test i prikazana je na slici 32. Za pokretanje testa iskorišćena je komanda prikazana na slici 33.

```
begin;
insert into t_gc(v) values (random()::text);
commit;
```

Slika 32 - Jednostavna skripta korišćena za testiranje

```
pgbench -c <clients> -j <threads> -T <time> -f script.sql gc_test
```

Slika 33 - Komanda iskorišćena za pokretanje testa

Značenje promenljivih vrednosti je sledeće:

- *<clients>* - broj klijenata koji izvršavaju upite (broj konkurentnih sesija sa bazom)
- *<threads>* - broj threadova koji pgbench koristi kako bi upravljao klijentima (sesijama)
- *<time>* - vreme (u sekundama) izvršenja testa.

Rezultati izvršenja testova prikazani su u tabeli 1.

Tabela 1 - Rezultat testiranja grupnog commit-a

| Threads | Clients | Time [s] | Ukupno procesiranih transakcija | Prosečna latencija | TPS       |
|---------|---------|----------|---------------------------------|--------------------|-----------|
| 1       | 1       | 30       | 68_664                          | 0.437              | 2_289.03  |
| 5       | 4       | 30       | 255_306                         | 0.587              | 8_512.52  |
| 10      | 4       | 30       | 571_558                         | 0.594              | 19_060.54 |
| 10      | 10      | 30       | 586_927                         | 0.511              | 19_568.06 |
| 20      | 4       | 30       | 947_336                         | 0.633              | 31_595.94 |
| 50      | 4       | 30       | 2_017_292                       | 0.743              | 67_337.88 |
| 100     | 4       | 30       | 2_631_241                       | 0.853              | 87_951.20 |

Rezultati pokazuju da sa porastom broja paralelnih klijenata značajno raste transakciona propusnost, dok prosečna latencija raste znatno sporije. Ovo ukazuje da PostgreSQL veoma efikasno obrađuje veliki broj kratkih konkurentnih transakcija. Takvo ponašanje je u skladu sa *group commit* mehanizmom, jer veći stepen konkurentnosti povećava verovatnoću da više transakcija bude potvrđeno kroz manji broj WAL flush operacija. Naravno, rezultat ovog merenja nije savršen. Na istoj mašini se izvršava baza, pgbench uz brojne druge aplikacije. Na pozitivan rezultat utiče mogu da utiču i faktori kao što su keširanje, upravljanje *lock-ovima* itd.

#### 4.4.2.2 Commit after insert VS batch insert VS copy

Ideja kod ovog testa je upoređivanje ponašanja baze podataka kada se za upis podataka koristi veći broj insert naredbi, pri čemu se vrši sinhroni *commit* nakon svake, batch insert, kod koga se veći broj insert naredbi grupiše i *copy* klauzula. Za testiranje biće korišćena tabela kreirana korišćenjem sledeće naredbe: `create table t_insert_test (id bigserial primary key, k integer, v text);`. U ovu tabelu će se upisivati 10\_000\_000 redova. Nakon izvršenja pojedinačnog testa baza podataka će biti ispražnjena, tako da će se svaki *test case* odvijati u sličnim uslovima sa stanovišta baze.

Za testiranje slučaja commit after insert biće iskorišćen *pgbench* i skripta prikazana na slici 34. *Pgbench* će SQL sa slike izvršiti 500\_000 puta, gde se *i* povećava za 1 u svakoj iteraciji. Batch insert će biti testiran na sličan način, samo što će insert naredbe biti grupisane u grupe od po 10 insert-a, a *pgbench* će skriptu izvršiti 50\_000 puta. Na kraju *COPY* – najpre će biti generisan fajl od 500\_000 redova, a zatim će biti pozvana *COPY* komanda nad bazom.

```
\set i random(1, 10000000)
begin;
insert into t_insert_test(k, v) values (:i, :i::text);
commit;
```

Slika 34 - Jednostavna skripta za potrebe testa 4.4.2.2

Tabela 2 - Rezultat testiranja optimizacije commit-a

|                           | Time [s] | WAL usage [MB] |
|---------------------------|----------|----------------|
| Commit after every insert | 209      | 90             |
| Batch insert              | 39       | 73             |
| COPY                      | 0.83     | 47.6           |

Tokom testa mereno je vreme koje je bilo potrebno da se podaci upišu u bazu, kao i količina generisanih *WAL* zapisa. Veličinu *WAL* zapisa moguće je odrediti izvršavanjem funkcije *pg\_current\_wal\_lsn*, kako bi se dobio trenutni *lsn* iz *WAL*-a, pre i posle upisa u bazu. Nakon toga se može upotrebiti funkcija *pg\_wal\_lsn\_diff* kako bi se odredila razlika (u bajtovima) između dva *lsn*.

Rezultati izvršenja testa su očekivani, ubedljivo najsporiji pristup je korišćenje *commit-a* nakon svake insert naredbe, pri čemu je i veličina generisanih *WAL* zapisa najveća. Na suprotnom kraju spektra, *COPY* naredba je za uvoz 500\_000 redova potrošila manje od 1 sekunde, a *WAL* potrošnja je skoro duplo manja nego prethodno pomenuti slučaj.

#### 4.4.3 Read path

*Read path* je, izuzev koraka planiranja izvršenja, jednostavniji od *write path-a*. U ovom praktičnom delu biće prikazana razlika između *cold* i *warm* čitanja. Pod *cold read* podrazumeva se slučaj kada se stranica na kojoj se nalazi podataka ne nalazi u kešu (u ovom slučaju u *shared buffers*) pa je baza primorana da najpre pribavi stranicu sa diska. Pod *hot read* podrazumeva se situacija u kojoj se stranica sa koje se čita podatak već nalazi u *shared buffers*, čime se štedi skupo čitanje sa diska. Za ovaj test biće upotrebljena tabela koja je kreirana za potrebe testa 4.4.1 i popunjena tokom izvršenja tog testa.

```

demo=# explain (analyze, buffers) select * from app_user where id = 2300;
                                QUERY PLAN
-----
Index Scan using app_user_pkey on app_user (cost=0.29..8.30 rows=1 width=21) (actual time=0.032..0.033 rows=1.00 loops=1)
  Index Cond: (id = 2300)
  Index Searches: 1
  Buffers: shared read=3
Planning:
  Buffers: shared hit=40 read=22
Planning Time: 1.006 ms
Execution Time: 0.124 ms
(8 rows)

demo=# explain (analyze, buffers) select * from app_user where id = 2300;
                                QUERY PLAN
-----
Index Scan using app_user_pkey on app_user (cost=0.29..8.30 rows=1 width=21) (actual time=0.022..0.024 rows=1.00 loops=1)
  Index Cond: (id = 2300)
  Index Searches: 1
  Buffers: shared hit=3
Planning Time: 0.095 ms
Execution Time: 0.064 ms
(6 rows)

```

Slika 35 - *Cold vs Warm read*

Na slici 35. prikazano je izvršenje istog upita u kratkom vremenskom intervalu. Radi se o jednosotavnoj pretrazi po primarnom ključu, nad kojim postoji indeks. Na osnovu rezultata izvršenja naredbe `explain (analyze, buffers)` može se primetiti da je *cold read*, odnosno prvo izvršenje već u fazi planiranja pristupilo velikom broju slotova u *shared buffers*, pri čemu su 22 slotova pribavljena sa diska. *Warm read* je sa druge strane pristupio slotovima u *shared buffers* samo 3 puta, što odgovara čitanju stranice sa meta podacima o indeksu, čitanje lista indeksa i čitanje stranice u kojoj se nalazi sam podatak.

## 5 Upravljanje baferima

U prethodnom poglavlju prikazan je način pristupa podacima koji PostgreSQL koristi prilikom upisa i čitanja. Za postizanje visokih performansi, ključan je korak u kome se stranice sa diska keširaju u deljivoj memoriji korišćenjem u okviru *shared buffers-a*. Iz konteksta *read* i *write path-a*, odnosno gledano na ovom nivou apstrakcije, ovaj pristup izgleda jednostavno – ukoliko se pristupa (bilo radi upisa ili čitanja) stranici koja je keširana u nekom slotu u *shared buffers*, jednostavno joj se pristupi, u suprotnom pribaviti je sa diska. Međutim, u pozadini ovaj proces je dosta komplikovaniji i prošao je decenije optimizacija i usavršavanja. Ključna okolnost koja ovaj proces čini u toj meri složenim je konkurentan pristup. Stoga, u ovom poglavlju, će fokus biti usmeren na detaljno prikazivanje kako PostgreSQL upravlja baferima.

### 5.1 Osnovni prikaz

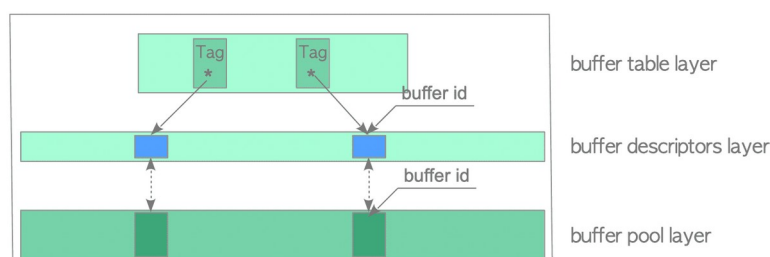
Već je više puta u toku rada objašnjeno kako PostgreSQL skladišti podatke u blokovima (stranicama) veličine 8KB. U prethodnom poglavlju videli smo kako se ovim stranicama pristupa, tačnije kako se ove starnice keširaju u RAM memoriji. Očigledno je da broj stranica koji može da stane u memoriju znatno manji nego broj stranica koje mogu da se čuvaju na disku. Zbog toga Postgre uvodi još dve strukture podataka: *buffer lookup hash table* i *buffer descriptor layer*. [17] Za upravljanje pristupu baferima zadužen je *buffer manager*.

Svakoj stranici se može dodeliti jedinstveni tag, koji se naziva *buffer tag*. Njegova definicija prikazana je na slici 36.

```
typedef struct buftag
{
    Oid          spcOid;          /* tablespace oid */
    Oid          dbOid;          /* database oid */
    RelFileNumber relNumber;     /* relation file number */
    ForkNumber   forkNum;        /* fork number */
    BlockNumber  blockNum;       /* blknum relative to begin of reln */
} BufferTag;
```

Slika 36 - definicija *buftag-a*

Polje *spcOid* označava *tablespace* kome pripada relacija koja sadrži stranicu, *dbOid* označava bazu kojoj pripada relacija, a *relNumber* je identifikator same relacije kojoj pripada stranica. Polje *blockNum* označava redni broj stranice unutar relacije, a *forkNum* označava da li se radi o stranici sa podacima, *freespace* mapi ili *visibility* mapi.

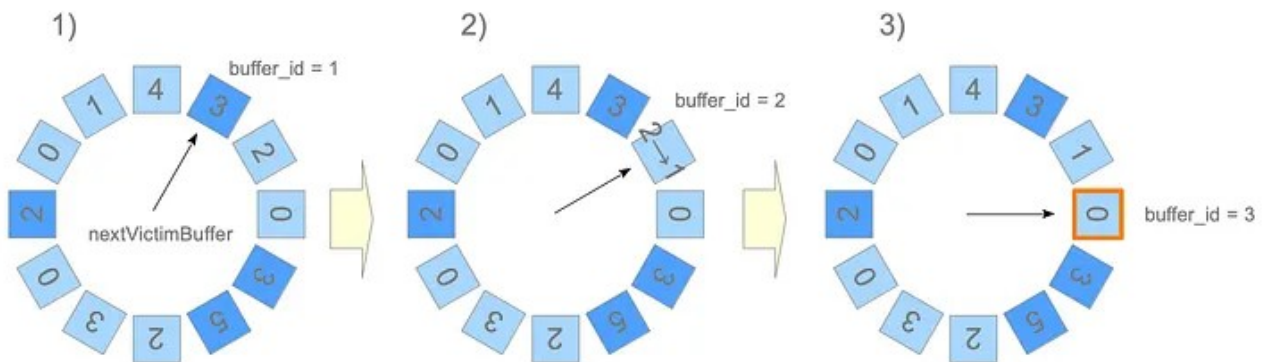


Slika 37 - Strukture koje čine shared buffers, preuzeto iz [18]

Na slici 37 prikazan je skup struktura koje čine *shared buffers* deo memorije. Komponente su sledeće:

- *Buffer pool* – niz koji skladišti stranice. Svakom slotu se pristupa na osnovu *buffer\_id*-ja.
- *Buffer descriptors* – svaki deskriptor ima odgovarajući slot u *buffer pool*-u, i sadrži metapodatke o tom slotu
- *Buffer table* – hash tabela koja čuva mapiranje između *buffer tag*-a i *buffer\_id*-ja

Kada proces želi da pribavi neku stranicu pristupa joj na osnovu *buffer tag*-a. *Buffer manager* prvo proverava u *buffer* tabeli da li postoji ključ sa zahtevanim tagom. Ukoliko postoji, stranica se čita direktno iz *buffer pool*-a. U suprotnom, stranica se čita sa diska. Ukoliko je *buffer pool* pun, potrebno je odrediti stranicu koja će biti izbačena kako bi se napravilo mesto za stranicu koju je potrebno pročitati sa diska. [17] U tu svrhu, PostgreSQL koristi *clock sweep* algoritam zamene stranica. Na slici 38 ilustrativno je dat prikaz ovog algoritma.



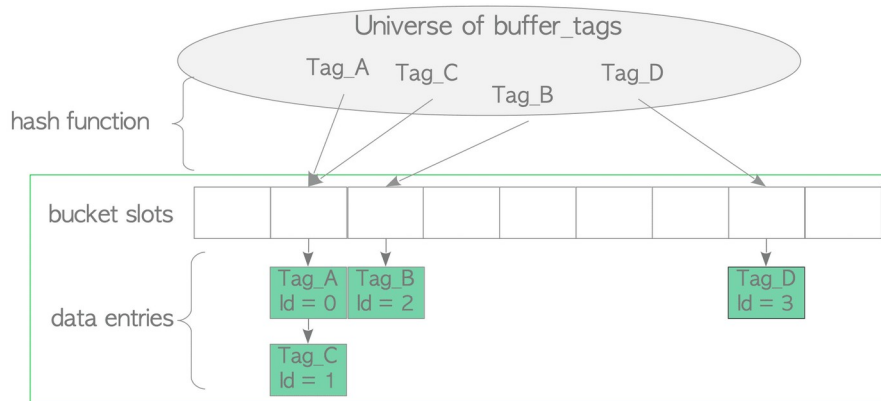
Slika 38 - *Clock sweep* algoritam, preuzeto iz [17]

Implementacija ovog algoritma u PostgreSQL podrazumeva postojanje pokazivača *nextVictimBuffer* koji se pomera slično kazaljki na satu. Ovaj pokazivač ukazuje na element u *buffer descriptor* nizu. Jedno od polja koje svaki deskriptor u nizu sadrži je *usageCount*. Kada se pristupi stranici, *usageCount* se povećava za 1. Kada je potrebno izabrati stranicu koja će biti žrtvovana pokazivač se pomera za jedno mesto sve dok ne pronade deskriptor koji ima *usageCount* 0. Pored toga, svaki deskriptor preko koga pokazivač pređe smanjuje *usageCount* za 1. [17]

## 5.2 Buffer table

*Buffer table* je heš tabela koja se koristi za mapiranje *buffer tag*-a u *buffer\_id*. Do verzije 8.2 ova struktura podataka je bila implementirana kao u ne *thread safe* maniru, pa je pristup celoj hash tabeli bio čuvan korišćenjem jednog *lock*-a. U nekom trenutku ovakva implementacija je postala glavno usko grlo kod upravljanja baferima pa je implementacija izmenjena kako bi se povećala granularnost zaključavanja. Moderna implementacija je vrlo slična implementaciji konkrutnih heš mapi u različitim programskim jezicima. [17]

Logički, ova tabela može da se podeli na tri dela – heš funkcija, heš *bucket*-i i sami elementi koji se skladište u mapi. Heš funkcija mapira *buffer tag* u *bucket*. Elementi koji se mapiraju u isti *bucket* su ulančani korišćenjem lančane liste. Prikaz strukture *buffer* tabele prikazan je na slici 39. Ovakva struktura bafer tabele omogućava da se prilikom pristupa zaključa samo jedan *bucket*, a ne cela heš tabela.



Slika 39 - Struktura *buffer table-a*, preuzeto iz [18]

## 5.3 Buffer descriptor

```
#define BM_DIRTY          (1 << 0)    /* data needs writing */
#define BM_VALID         (1 << 1)    /* data is valid */
#define BM_TAG_VALID     (1 << 2)    /* tag is assigned */
#define BM_IO_IN_PROGRESS (1 << 3)    /* read or write in progress */
#define BM_IO_ERROR      (1 << 4)    /* previous I/O failed */
#define BM_JUST_DIRTIED  (1 << 5)    /* dirtied since write started */
#define BM_PIN_COUNT_WAITER (1 << 6) /* have waiter for sole pin */
#define BM_CHECKPOINT_NEEDED (1 << 7) /* must write for checkpoint */
#define BM_PERMANENT     (1 << 8)    /* permanent relation (not unlogged) */
```

```
src/include/storage/buf_internals.h
typedef struct BufferDesc
{
    BufferTag    tag;                /* ID of page contained in buffer */
    BufFlags    flags;              /* see bit definitions above */
    uint16      usage_count;        /* usage counter for clock sweep code */
    unsigned    refcount;           /* # of backends holding pins on buffer */
    int         wait_backend_pid;   /* backend PID of pin-count waiter */
    slock_t     buf_hdr_lock;       /* protects the above fields */
    int         buf_id;             /* buffer's index number (from 0) */
    int         freeNext;           /* link in freelist chain */

    LWLockId    io_in_progress_lock; /* to wait for I/O to complete */
    LWLockId    content_lock;        /* to lock access to buffer contents */
} BufferDesc;
```

Slika 40 - Definicija *buffer descriptor-a*

Na slici 40 prikazana je definicija *buffer descriptor-a* u verziji PostgreSQL 9.5. U nastavku je dat detaljniji opis polja koji čine ovu strukturu. [18]

- *tag* – sadrži *buffer\_tag* stranice koja se nalazi u povezanom slotu u *buffer pool-u*.
- *buf\_id* – sadrži indeks, koji se takođe koristi za pristup slotu u *buffer pool-u*.
- *refcount* – takođe poznat i pod nazivom *pin count*, radi se o broju procesa koji trenutno pristupaju stranici iz *pool-a* koja je povezana sa deskriptorom. Kada proces pristupi stranici, ovo polje se inkrementira. Nakon što je pristup završen, vrednost se dekrementira. Kada je *refcount* 0, kaže se da je povezana stranica *unpinned*, u suprotnom je *pinned*.
- *usage\_count* – broj puta koliko je povezanoj stranici pristupano od kako je učitana u bafer



- *content\_lock* i *io\_in\_progress\_lock* – *lightweight* lockovi kojima se kontroliše pristup povezanoj stranici
- *buf\_hdr\_lock* – *spin lock* koji kontroliše pristup poljima *flags*, *usage\_count* i *refcount*
- *freeNext* – pokazivač na deskriptor za pristup *freelist* stranici.

PostgreSQL dugo vremena nije koristio atomične tipove podataka. Obzirom da se radi o bazi podataka koja treba da bude u mogućnosti da se izvršava na bilo kojoj mašini, pod bilo kojim operativnim sistemom, bilo je teško napraviti odgovarajuću apstrakciju koja bi pokrila sve moguće platforme. Tek su od verzije 9.5 uvedeni u nekim aspektima, a nakon toga u potpunosti ušli u upotrebu od verzije 9.6. Kao posledica toga, struktura prikazana na slici 40 promenjena je, pa je njen današnji izgled prikazan na slici 41. [17]

```
#define BM_LOCKED      (1U << 22) /* buffer header is locked */
#define BM_DIRTY      (1U << 23) /* data needs writing */
#define BM_VALID      (1U << 24) /* data is valid */
#define BM_TAG_VALID   (1U << 25) /* tag is assigned */
#define BM_IO_IN_PROGRESS (1U << 26) /* read or write in progress */
#define BM_IO_ERROR    (1U << 27) /* previous I/O failed */
#define BM_JUST_DIRTIED (1U << 28) /* dirtied since write started */
#define BM_PIN_COUNT_WAITER (1U << 29) /* have waiter for sole pin */
#define BM_CHECKPOINT_NEEDED (1U << 30) /* must write for checkpoint */
#define BM_PERMANENT    (1U << 31) /* permanent buffer (not unlogged,
| | | | | | | * or init fork) */

#define PG_HAVE_ATOMIC_U32_SUPPORT
typedef struct pg_atomic_uint32
{
    volatile uint32 value;
} pg_atomic_uint32;

typedef struct BufferDesc
{
    BufferTag    tag;          /* ID of page contained in buffer */
    int         buf_id;       /* buffer's index number (from 0) */

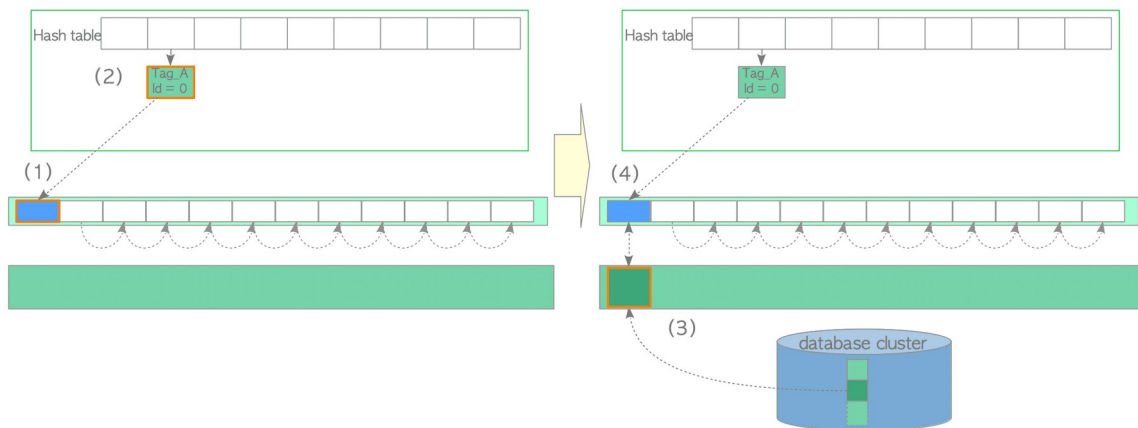
    /* state of the tag, containing flags, refcount and usagecount */
    pg_atomic_uint32 state;

    int         wait_backend_pgprocno; /* backend of pin-count waiter */
    int         freeNext;             /* link in freelist chain */
    PgAioWaitRef io_wref;             /* set iff AIO is in progress */
    LWLock      content_lock;         /* to lock access to buffer contents */
} BufferDesc;
```

Slika 41 - Moderni izgled strukture deskriptora

Razlika u odnosu na prethodnu strukturu je u tome što su polja *flags*, *refcount* i *usagecount* grupisana u jedan atomičan podatak. Kao posledica ovoga, lockovi *io\_in\_progress\_lock* i *buf\_hdr\_lock* su izbačeni, obzirom da više nije potrebno čuvati pristup objedinjenim poljima. [18]

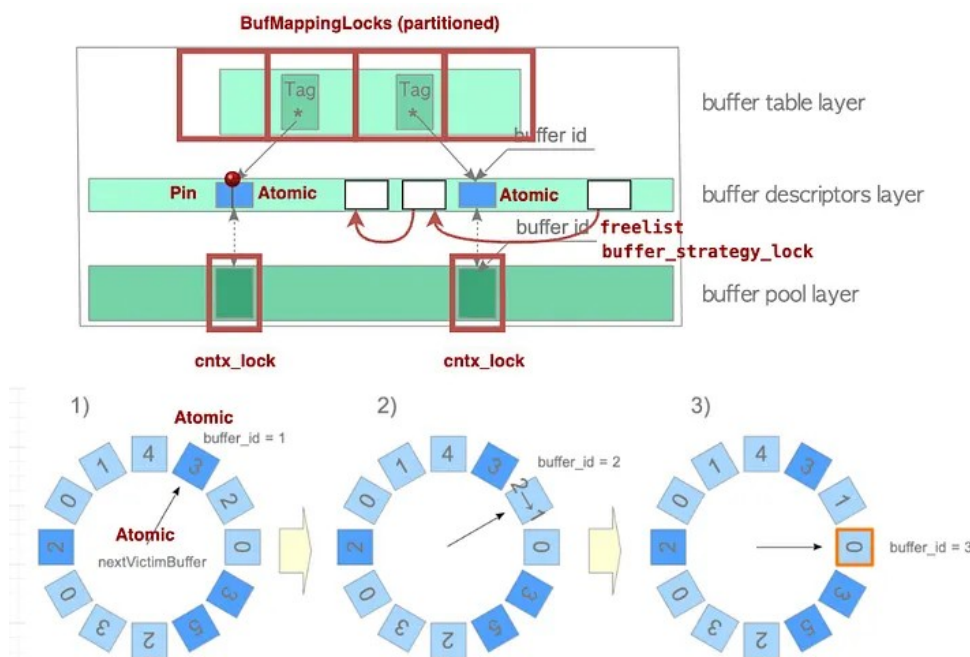
Inicijalno, nakon pokretanja PostgreSQL servera, svi bafer deskriptori su prazni. Svi prazni bafer deskriptori čine *freelist*. Kada se prva stranica učitava u memoriju, najpre se pribavi odgovarajući deskriptor i pinuje se (inkrementira se *refcount* i *usage\_count*). Nakon toga se insertuje novi element u bafer tabelu, a zatim se učitava stranica u *bafer pool* i na kraju se ažuriraju metapodaci u deskriptoru. [18] Na slici 42 ilustrativno je prikazan ovaj proces.



Slika 42 - Učitavanje stranice u memoriju

## 5.4 Buffer manager lock-ovi

Na slici 43 dat je prikaz *lock-ova* koji su u upotrebi u PostgreSQL-u od verzije 9.6.



Slika 43 - Lockovi koje se koriste za upravljanje baferima, preuzeto iz [17]

*BufMappingLocks* se koristi kako bi se obezbedio konkurentan pristup bafer tabeli. Radi se o klasičnom *reader-writer lock-u* koji omogućava da veći broj procesa čita podataka iz tabele, ali se *lock* mora pribaviti u ekskluzivnom modu kako bi upis/brisanje/ažuriranje bili mogući. Kao što je naglašeno u poglavlju 5.2 bafer tabela je podeljena na *bucket-e* pri čemu postoji jedan *BufMappingLock* za svaki *bucket* u tabeli.

Pre verzije 9.5 PostgreSQL je za pristup pojedinačnim deskriptorima koristio *buf\_hdr\_lock*, što je i prikazano sadržajem deskriptora. Pored *lock-a* za pojedinačne deskriptore, koristio se i *lock* nad celim deskriptor *layer-om* kada se izvršavao *clock sweep* algoritam. To znači da, kada se izvršavao algoritam za određivanje stranice koja će biti zamenjena, svi ostali pristupi baferima bili su blokirani. Ovakav pristup je zamenjen nakon uvođenja atomičnih promenljivih. Zaključavanje celog deskriptor *layera* zamenjeno je kratkim *buffer\_strategy\_lock-om* koji se pribavlja i oslobađa prilikom pristupa *freelist-i*, a sama kazaljka (*nextVictimBuffer* pokazivač) je postala atomičan ceo broj. [17]



Takođe, od verzije 9.6 PostgreSQL-a kao što je već napomenuto, pristup poljima *refcount*, *usage\_count* i *flags* u okviru pojedinačnog deskriptora oslanja se na atomične promenljive.

*Lock* koji kontroliše pristup samim stranicama u *buffer pool-u* naziva se *content lock*. Radi se o *reader-writer lock-u*, što znači da veći broj procesa može paralelno da pristupa stranici radi čitanja, ali *lock* mora da se pribavi u *exclusive* modu kako bi se stranica upisala. *Content lock* se razlikuje od zaključavanja deskriptora po svojoj svrsi. *Content lock* se koristi za upravljanje konkrutnim čitanjem i upisom u podataka u stranicu, dok se *lock-ovi* u okviru deskriptora koriste kako bi se na primer kontrolisao algoritam zamene stranica. Kod izvršenja ovog algoritma, proces ne mora da poseduje *content lock*, ali kako bi se pribavio *content lock*, proces najpre mora da pinuje odgovarajući bafer deskriptor. Veći broj procesa u jednom trenutku može da pinuje jedan bafer deskriptor, ali samo jedan od njih može da drži *content lock* u *exclusive* modu. [17]

## 6 WAL

Logovi kod baza podataka predstavljaju uređeni žurnal promena koje su se desile nad bazom podataka. Praktično sve relacione baze podataka implementiraju logove na neki način, a logovi predstavljaju standardni način za implementaciju ACID svojstava, naročito *durability* svojstva. *Durability* svojstvo garantuje da jednom kada je transakcija komitovana, njeni efekti su trajni, odnosno u slučaju pada baze ili sistema, podaci ostaju bezbedni. U slučaju da dođe do pada sistema, podaci iz logova se mogu iskoristiti kako bi se baza podataka vratila u konzistentno stanje, čime se čuva integritet podataka. [19] U svetu baza podataka postoje tri tipa logova:

- *WAL / Redo Log* – često se ova dva termina koriste naizmenično kako bi se opisao isti koncept. Osnovna ideja je da se promene upisuju u *log* pre nego što budu upisane u primarno skladište. Redo aspekt se odnosi na mogućnost da se promene koje su zapisane u logovima ponovo primene nad bazom tokom oporavka.
- *Undo Log* – logovi skladište informacije koje su neophodne kako bi se poništio efekat modifikacija. Primarno se koristi prilikom *rollback-a* transakcija i radi postizanja *read* konzistencije u nekim MVCC implementacijama. Koriste se kod Oracle i MySQL baze podataka radi implementacije mehanizma vidljivosti unutar transakcija. [19]

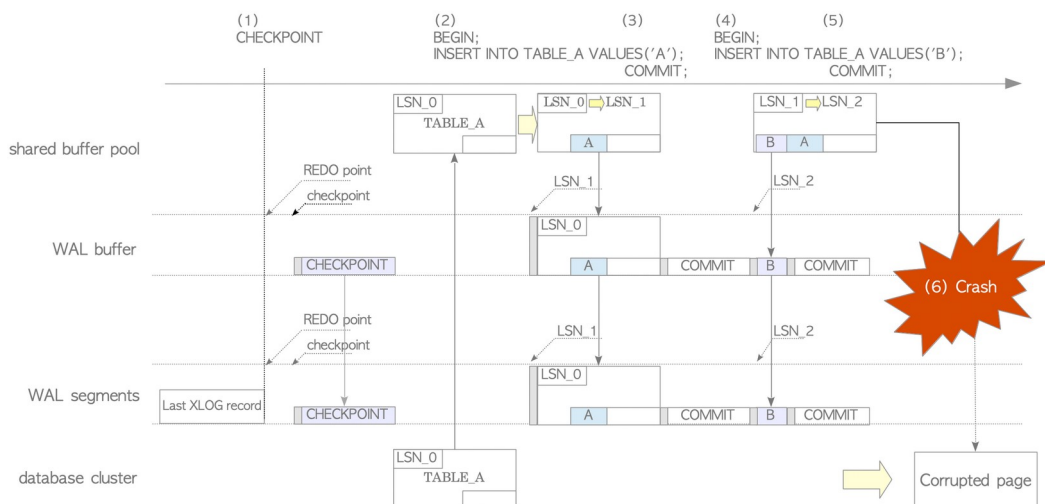
Drugi značajna prednost koju donose logovi je to što nije neophodno da se promene nad podacima odmah upisuju na disk. Takav način rada, kod koga se promene nad stranicom u memoriji odmah upisuju na disk je neuporedivo sporiji od pristupa koji koristi WAL. Kod PostgreSQL-a, WAL pruža mogućnost *on-line backup-a* kao i *point-in-time recovery*. [20] U ovom poglavlju će detaljno biti opisan način rada *WAL-a* kod PostgreSQL-a, pri čemu će fokus biti na načinu na koji se generišu WAL zapisi i skladištenju istih. Oporavak baze i povezani mehanizmi neće biti razmatrani jer izlaze iz okvira ovog rada.

### 6.1 Osnovni pojmovi

Kao što je već mnogo puta rečeno, PostgreSQL upisuje sve modifikacije nad podacima u *WAL*. Ovi podaci nazivaju se *XLOG record(s)* ili *WAL podaci*. *XLOG records* se najpre upisuju u *WAL buffer* u deljivoj memoriji, nakon čega su odmah upisuju u *WAL segment file* na disku. Kao što je naglašeno u poglavlju 2.5, ovi fajlovi se nalaze u *pg\_wal/* poddirektorijumu. *LSN (Log Sequence Number)* *XLOG record-a* predstavlja lokaciju gde je *record* upisan u transakcioni log. Takođe, koristi se kao jedinstveni identifikator *WAL* zapisa. Prilikom oporavka baze podataka, tačka od koje je potrebno *replay-ovati* *WAL* zapise naziva se *REDO point*. *REDO point* se upisuje u *WAL* prilikom *checkpoint* akcije. PostgreSQL podržava i mogućnost *full-page* zapisa. Radi se o *XLOG recordu* koji sadrži ceo *header* i sadržaj stranice. Ova funkcionalnost omogućava i zaštitu i oporavak u slučaju da je došlo do korupcije podataka zbog greške operativnog sistema, naravno po cenu dosta većih *WAL* fajlova. [21]

Osnovni tok koji se prolazi prilikom upisa u *WAL* prikazan je na slici 44. Kada se izda *INSERT* naredba, PostgreSQL najpre učitava odgovarajuću stranicu tabele *TABLE\_A* u memoriju, upisuje odgovarajući *tuple* u tu stranicu i generiše *XLOG* zapis ove akcije koji se upisuje u *WAL buffer*. Nakon toga se *pd\_lsn header* polje stranice postavlja na *LSN\_1*. Kada se transakcija komituje, generiše se još jedan *XLOG* zapis koji se upisuje u *WAL buffer* nakon čega se podaci iz njega *flush-*

uju na disk, u odgovarajući segment fajl, počevši od *LSN\_1*. Identičan niz koraka se prolazi nakon što se izda sledeća naredba za upis podatka u tabelu. [21]



Slika 44 - Uprošćen prikaz upisa u WAL, preuzeto iz [21]

## 6.2 Struktura WAL segment fajlova

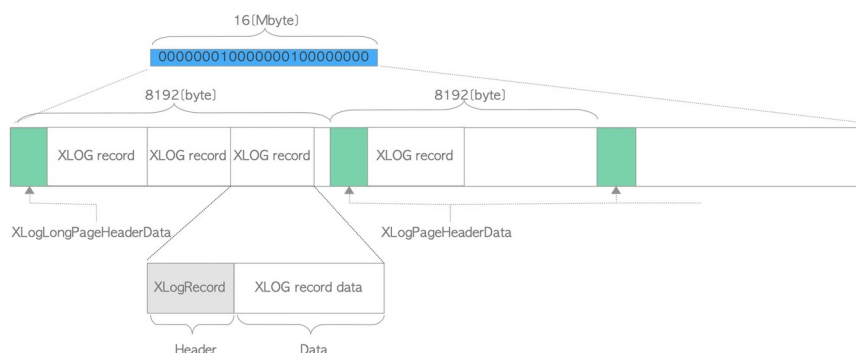
PostgreSQL WAL posmatra kao virtuelni fajl koji je 8 bajtova dugačak (što znači da ima  $2^{64}$  raspoloživih adresa). Obzirom da ovo prevazilazi realne mogućnosti skladištenja fajlova, log je razbijen na WAL segmente, pri čemu je svaki 16MB. Formiranje imena WAL segment fajla prikazano je na slici 45.

$$\text{WAL segment file name} = \text{timelineId} + (\text{uint32}) \frac{\text{LSN} - 1}{16\text{M} * 256} + (\text{uint32}) \left( \frac{\text{LSN} - 1}{16\text{M}} \right)$$

Slika 45 - Formiranje imena WAL segment fajla, preuzeto iz [21]

Polje *timelineId* je tipa *unsigned integer* i koristi se kod *point-in-time* oporavka. Prvi WAL segment je 00000001 00000000 00000001. Kada se on ispuni do kraja, sledeći dobija naziv 00000001 00000000 00000002. Nakon fajla koji ima ime 00000001 00000000 000000FF, sledeći dobija ime 00000001 00000001 00000000. Pravilo je da kada se desi overfčpw poslednje dve cifre, srednji segment se povećava za 1. [21]

Kao i stranice koji sadrže podatke na disku, jedan WAL segment je podeljen na stranice iste veličine (8KB). Svaka stranica sadrži *header*, pri čemu se *header* prve stranice u WAL segmentu razlikuje od *header-a* preostalih stranica. Na slici 46 ilustrativno je prikazan izgled WAL segmenta.



Slika 46 - Izgled WAL segmenta, preuzeto iz [21]

Na slikama 47 i 48 prikazana je definicija *header-a* početne stranice u segmentu i definicija *header-a* ostalih stranica.

```
typedef struct XLogPageHeaderData
{
    uint16      xlp_magic;          /* magic value for correctness checks */
    uint16      xlp_info;          /* flag bits, see below */
    TimeLineID  xlp_tli;          /* TimeLineID of first record on page */
    XLogRecPtr  xlp_pageaddr;      /* XLOG address of this page */

    /*
     * When there is not enough space on current page for whole record, we
     * continue on the next page. xlp_rem_len is the number of bytes
     * remaining from a previous page; it tracks xl_tot_len in the initial
     * header. Note that the continuation data isn't necessarily aligned.
     */
    uint32      xlp_rem_len;       /* total len of remaining data for record */
} XLogPageHeaderData;
```

Slika 47 - Definicija *header-a* ostalih stranica u segmentu

```
/*
 * When the XLP_LONG_HEADER flag is set, we store additional fields in the
 * page header. (This is ordinarily done just in the first page of an
 * XLOG file.) The additional fields serve to identify the file accurately.
 */
typedef struct XLogLongPageHeaderData
{
    XLogPageHeaderData std;        /* standard header fields */
    uint64      xlp_sysid;        /* system identifier from pg_control */
    uint32      xlp_seg_size;     /* just as a cross-check */
    uint32      xlp_xlog_blkisz;  /* just as a cross-check */
} XLogLongPageHeaderData;

#define SizeOfXLogLongPHD    MAXALIGN(sizeof(XLogLongPageHeaderData))

typedef XLogLongPageHeaderData *XLogLongPageHeader;

/* When record crosses page boundary, set this flag in new page's header */
#define XLP_FIRST_IS_CONTRCORD    0x0001
/* This flag indicates a "long" page header */
#define XLP_LONG_HEADER          0x0002
/* Replaces a missing contrecord; see CreateOverwriteContrecordRecord */
#define XLP_FIRST_IS_OVERWRITE_CONTRCORD 0x0004
/* All defined flag bits in xlp_info (used for validity checking of header) */
#define XLP_ALL_FLAGS            0x0007
```

Slika 48 - Definicija *header-a* prve stranice u segmentu

## 6.3 Struktura XLOG record-a

Kao što se može videti sa slike 46, svaki *XLOG record* se sastoji iz *header-a* i podataka. Definicija *headera* prikazana je na slici 49. Većina polja koja se nalaze u *header-u* imaju jasnu funkciju. Najzanimljivija polja su *xl\_info* i *xl\_rmid*. Radi se o poljima koja su povezana sa *resource manager-om*. *Resource manager* definiše operaciju koja će se primeniti kada se konkretan *XLOG* zapis upisuje u *WAL* ili se primenjuje prilikom oporavka. Primer *resource manager-a* je *RM\_HEAP*, *RM\_BTREE*, *RM\_SEQ*, *RM\_TBLSPC*... [21]

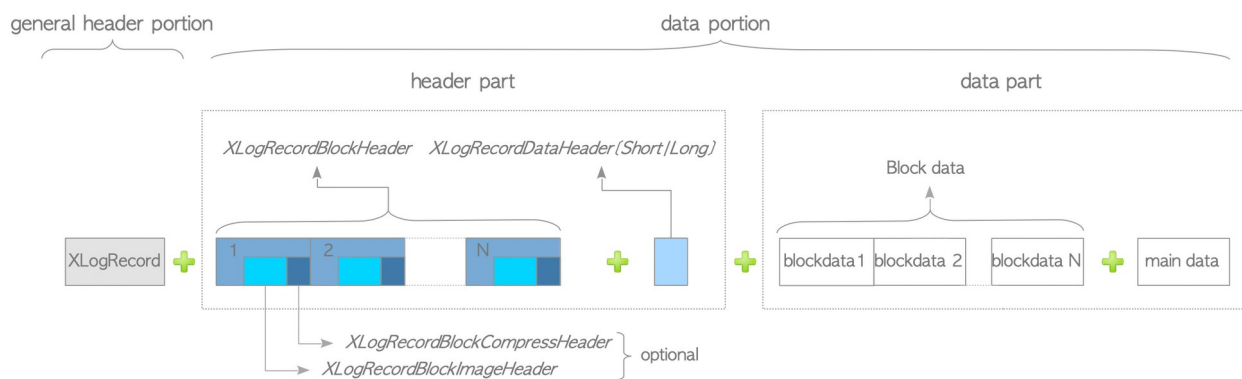
```

typedef struct XLogRecord
{
    uint32      xl_tot_len;      /* total len of entire record */
    TransactionId xl_xid;        /* xact id */
    XLogRecPtr   xl_prev;        /* ptr to previous record in log */
    uint8        xl_info;        /* flag bits, see below */
    RmgrId       xl_rmid;        /* resource manager for this record */
    /* 2 bytes of padding here, initialize to zero */
    pg_crc32c    xl_crc;        /* CRC for this record */

    /* XLogRecordBlockHeaders and XLogRecordDataHeader follow, no padding */
} XLogRecord;

```

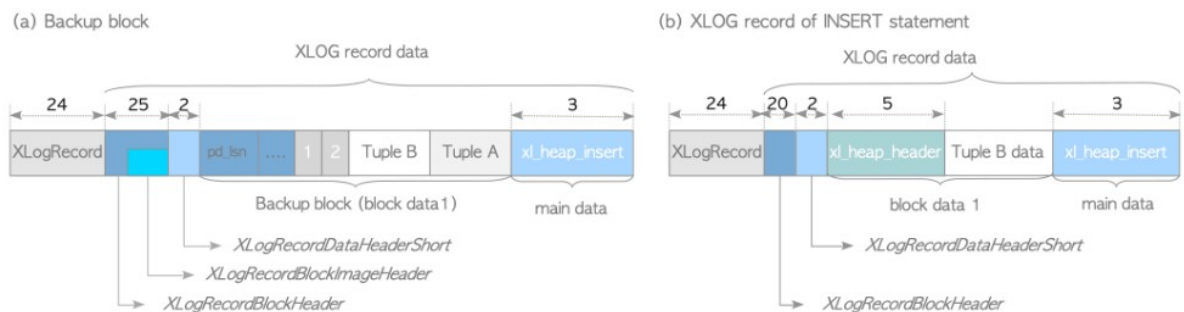
Slika 49 - Definicija header-a XLOG record-a



Slika 50 - Izgleda data dela XLOG zapisa, preuzeto iz [21]

Ilustracija data dela XLOG zapisa prikazana je na slici 50. Data deo XLOG record-a se takođe može podeliti na dva dela – header i data. Header deo se sastoji od nula ili više XlogRecordBlockHeaders- i nula ili jednog XlogRecordDataHeaderShort, pri čemu mora da sadrži bar jedan od njih. Kada se skladišti full-page zapis XlogRecordBlockHeader sadrži XlogRecordBlockImageHeader i XlogRecordBlockCompressHeader. Definicija svih ovih header-a je izostavljena, a sve definicije dostupne su u [22].

Što se tiče data dela data dela XLOG zapisa, on se sastoji od nula ili više blockdata blokova i nula ili jednog main data bloka. Block data deo je deo u kome se nalazi konkretna promena koja se desila u bazi, zajedno sa lokacijom gde se desila. Main data deo je deo koji se odnosi na ceo XLOG zapis i predstavlja dodatne podatke koje beleži resource manager. Konkretna primer XLOG zapisa za proste operacije biće prikazan u praktičnom delu ovog poglavlja, a na slici 51 prikazan je izgled nekih zapisa. Struktura zapisa se dodatno komplikuje ukoliko se uključi logička replikacija.



Slika 51 - Struktura nekih XLOG zapisa, preuzeto iz [21]

## 6.4 Upravljanje WAL-om

Dosadašnji deo ovog poglavlja je u primarni fokus stavljao detalje niskog nivoa o tome kako WAL skladišti podatke. Taj nivo poznavanja detalja od koristi je za ljude koji rade na razvoju i održavanju PostgreSQL-a, ili alata koji olakšavaju upravljanje PostgreSQL-om. Sa stanovišta programera ili *database* administratora daleko je značajnije znanje o podešavanjima koja direktno utiču na rad baze podataka. U ostatku ovog poglavlja biće dat prikaz parametara i drugih aspekata koji utiču na rad WAL-a.

Parametri koji utiču na rad WAL-a su:

- *wal\_block\_size* – parametar koji kontroliše veličinu stranice u WAL segmentu. Prilikom kompajliranja baze, njegova vrednost je postavljena na 8KB i nije je moguće promeniti.
- *wal\_buffers* – ovaj parametar je već pomenut u poglavlju 4.1.2, a on kontroliše veličinu prostora u okviru deljive memorije gde se XLOG zapisi privremeno upisuju pre nego što se *flush-uju* na disk. *Default* veličina je 4MB. Manja veličina ovog parametra znači da će PostgreSQL češće vršiti upis na disk, čime se povećava broj IO operacija.
- *wal\_compression* – ovaj parametar kontroliše da li će *full page XLOG* zapisi biti kompresovani pre upisa u segment. U modernim sistemima, naročito bazi podataka, CPU ne predstavlja usko grlo, pa kompresovanje donosi značajno poboljšanje u vidu veličine WAL segmenata. [23]
- *wal\_decode\_buffer\_size* – ovaj parametar kontroliše veličinu *chunk-a* iz WAL-a koji PostgreSQL čita prilikom logičke replikacije ili prilikom oporavka. Postavljanje nešto veće vrednosti smanjuje broj *disk IO* operacija koje se izvrše
- *wal\_keep\_size* – ovim parametrom se kontroliše koliko WAL-a treba da se zadrži kako bi se podržali replika serveri. Ukoliko je ovaj parametar 0, replikacija može da prestane sa radom jer su WAL segmenti obrisani pre nego što je *standby* server uspeo da ih konzumira. [23]
- *wal\_level* – postoje tri moguće vrednosti koje ovaj parametar može imati – *minimal*, *replica*, *logical*. Kada je *wal\_level* postavljen na *minimal* ne postoji podrška za bilo koji vid replikacije, *replica level* dozvoljava fizičku replikaciju, a *logical level* omogućava logičku replikaciju.
- *wal\_recycle* – umesto brisanja starih WAL fajlova i kreiranja novih, PostgreSQL može da reciklira stare fajlove tako što im promeni ime.
- *wal\_segment\_size* – kontroliše veličinu segment fajlova. Prednosti postoji i u slučaju manjih i u slučaju većih fajlova. Kod većih fajlova smanjuje se ukupan broj fajlova i poboljšava se sekvencijalni IO pristup. Kod manjih fajlova lakše je arhiviranje sa većom granularnošću kao i lakše upravljanje WAL-om.
- *wal\_skip\_threshold* – ovim parametrom se definiše kada će PostgreSQL preskočiti upis WAL-a za velike operacije i umesto toga direktno sinhronizovati same stranice sa podacima.
- *wal\_sync\_method* – ovo podešavanje se odnosi na mogućnosti koje su opisane u poglavlju 4.2.1
- *wal\_writer\_delay* – definiše koliko često se vrši *flush* zapisa iz bafera u memoriji na disk

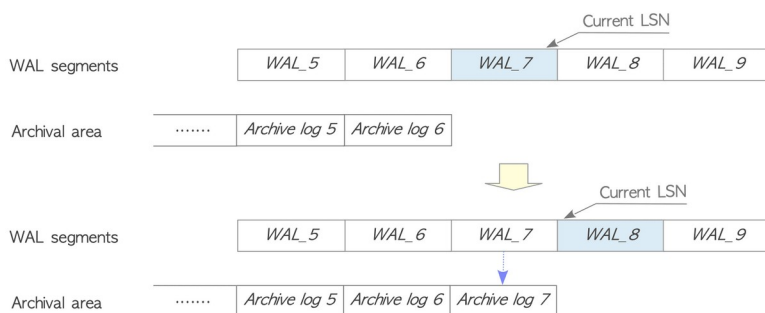
Pored prikazanih parametara postoji još jedan deo parametara, koji se prevashodno odnose na replikaciju i kojima se kontrolišu različiti aspekti iste.

Konfiguracija WAL-a nije usmerena samo na način upisa *XLOG* zapisa, već pre svega balans između performansi, količine generisanog WAL-a i vremena oporavka nakon pada sistema. Centralnu ulogu imaju *checkpoint-i*, jer oni određuju trenutak kada se izmenjene stranice podataka upisuju na disk i od kog mesta bi oporavak posle pada trebao da krene. Nakon *checkpoint-a* stariji WAL segmenti više nisu potrebni za *recovery* i mogu biti obrisani ili reciklirani, osim ako su potrebni za arhiviranje ili replikaciju. [24] Najvažniji parametri koji kontrolišu *checkpoint* proces su:

- *checkpoint\_timeout* – maksimalno vreme između automatski *WAL checkpoint-a*
- *max\_wal\_size* – maksimalna veličina do koje WAL može da raste pre automatskog *checkpoint-a*
- *checkpoint\_completion\_target* – određuje koliko ravnomerno PostgreSQL raspoređuje *checkpoint IO* operacije tokom intervala između dva *checkpoint-a*.

Osnovna ideja iza *checkpoint-a* je da se u svakom trenutku čuvaju samo oni WAL segment fajlovi koji su neophodni za oporavak baze.

PostgreSQL nudi mogućnost kontinualnog arhiviranja WAL segmenata. Osnovna ideja je da svaki puta kada se desi *segment switch*, odnosno promena segment fajla u koji se upisuju zapisi, stari segment fajl se arhivira i upisuje na *archive log* lokaciju. [21] Ilustracija kontinualnog arhiviranja prikazana je na slici 52.



Slika 52 - Kontinualno arhiviranje, preuzeto iz [21]

## 6.5 Praktični prikaz

U praktičnom delu ovog poglavlja, najpre će biti prikazan izgleda *XLOG* zapisa koji su posledica jednostavne operacije, kao što je *INSERT*. Nakon toga, drugi deo će se odnositi na ispitivanje uticaja pojedinih akcija i podešavanja na ponašanje WAL-a.

### 6.5.1 XLOG records

Za potrebe ovog testa kreirana je u potpunosti nova instanca PostgreSQL-a, kao i baza unutar te instance *wal\_test*. Za potrebe testiranja biće korišćena ekstenzija *pg\_walinspect* koja omogućava ispitivanje sadržaja WAL-a na niskom nivou, kao i alat *pg\_waldump* koji služi kako bi se sadržaj WAL-a prikazao u formatu koji je razumljiv čoveku. Za potrebe demonstracije kreirana je tabela koja je prikazana na slici 10. Pre izvršenja bilo kakvih upita izvršen je upit `select pg_current_wal_lsn();` kako bi se pribavio *lsn* što će olakšati pretragu WAL segment fajla. Na



slici 53 prikazan je deo sadržaja segment fajla dobijem korišćenjem *pg\_waldump* alata bez dodatnih filtriranja. Na slici 54 prikazana je serija operacija koje su izvršene neposredno pre izvršenja *pg\_waldump* operacije prikazane na slici 53.

```
lule@lule-Asus-Vivobook:~$ sudo -u postgres /usr/lib/postgresql/18/bin/pg_waldump -p /var/lib/postgresql/18/main/pg_wal 000000010000000000000001 | tail -10
pg_waldump: error: error in WAL record at 0/1C29BE0: invalid record length at 0/1C29C18: expected at least 24, got 0
rmgr: XLOG len (rec/tot): 49/ 8193, tx: 0, lsn: 0/01C22FF8, prev 0/01C22FC0, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/1249 blk 16 FPM
rmgr: XLOG len (rec/tot): 49/ 2917, tx: 0, lsn: 0/01C25018, prev 0/01C22FF8, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/2606 blk 4 FPM
rmgr: XLOG len (rec/tot): 49/ 8181, tx: 0, lsn: 0/01C25080, prev 0/01C25018, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/2610 blk 0 FPM
rmgr: XLOG len (rec/tot): 49/ 7913, tx: 0, lsn: 0/01C27B90, prev 0/01C25080, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/1249 blk 54 FPM
rmgr: Heap len (rec/tot): 70/ 70, tx: 767, lsn: 0/01C29A98, prev 0/01C27B90, desc: INSERT+INIT off: 1, flags: 0x00, blkref #0: rel 1663/16388/16396 blk 0
rmgr: Standby len (rec/tot): 54/ 54, tx: 0, lsn: 0/01C29AE0, prev 0/01C29A98, desc: RUNNING_XACTS nextXid 768 latestCompletedXid 766 oldestRunningXid 767; 1 xacts: 767
rmgr: Btree len (rec/tot): 90/ 90, tx: 767, lsn: 0/01C29B18, prev 0/01C29AE0, desc: NEWROOT level: 0, blkref #0: rel 1663/16388/16403 blk 1, blkref #2: rel 1663/16388/16403 blk 0
rmgr: Btree len (rec/tot): 64/ 64, tx: 767, lsn: 0/01C29B78, prev 0/01C29B18, desc: INSERT_LEAF off: 1, blkref #0: rel 1663/16388/16403 blk 1
rmgr: Transaction len (rec/tot): 34/ 34, tx: 767, lsn: 0/01C29B88, prev 0/01C29B78, desc: COMMIT 2026-04-19 13:19:03.566123 CEST
rmgr: Standby len (rec/tot): 50/ 50, tx: 0, lsn: 0/01C29BE0, prev 0/01C29B88, desc: RUNNING_XACTS nextXid 768 latestCompletedXid 767 oldestRunningXid 768
```

Slika 53 - Sadržaj segment fajla, dobijen korišćenjem *pg\_waldump* alata

```
wal_test=# select pg_current_wal_lsn();
pg_current_wal_lsn
-----
0/1C22FF8
(1 row)

wal_test=# insert into app_user values (1, 'Luka', 'zebra');
INSERT 0 1
wal_test=# select pg_current_wal_lsn();
pg_current_wal_lsn
-----
0/1C29BE0
(1 row)
```

Slika 54 - Izvršenje *INSERT* naredbe, uz prethodno i naknadno pribavljanje *LSN*-a

Sada kada smo pribavili *LSN* pre izvršenja *INSERT* naredbe i *LSN* nakon izvršenja naredbe možemo preciznije da pretražimo *WAL* segment kako bismo prikazali šta je sve upisano u *WAL* prilikom upisa podataka u tabelu.

```
lule@lule-Asus-Vivobook:~$ sudo -u postgres /usr/lib/postgresql/18/bin/pg_waldump -p /var/lib/postgresql/18/main/pg_wal -s 0/1C22FF8 -e 0/1C29BE0
rmgr: XLOG len (rec/tot): 49/ 8193, tx: 0, lsn: 0/01C22FF8, prev 0/01C22FC0, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/1249 blk 16 FPM
rmgr: XLOG len (rec/tot): 49/ 2917, tx: 0, lsn: 0/01C25018, prev 0/01C22FF8, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/2606 blk 4 FPM
rmgr: XLOG len (rec/tot): 49/ 8181, tx: 0, lsn: 0/01C25080, prev 0/01C25018, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/2610 blk 0 FPM
rmgr: XLOG len (rec/tot): 49/ 7913, tx: 0, lsn: 0/01C27B90, prev 0/01C25080, desc: FPI_FOR_HINT , blkref #0: rel 1663/16388/1249 blk 54 FPM
rmgr: Heap len (rec/tot): 70/ 70, tx: 767, lsn: 0/01C29A98, prev 0/01C27B90, desc: INSERT+INIT off: 1, flags: 0x00, blkref #0: rel 1663/16388/16396 blk 0
rmgr: Standby len (rec/tot): 54/ 54, tx: 0, lsn: 0/01C29AE0, prev 0/01C29A98, desc: RUNNING_XACTS nextXid 768 latestCompletedXid 766 oldestRunningXid 767; 1 xacts: 767
rmgr: Btree len (rec/tot): 90/ 90, tx: 767, lsn: 0/01C29B18, prev 0/01C29AE0, desc: NEWROOT level: 0, blkref #0: rel 1663/16388/16403 blk 1, blkref #2: rel 1663/16388/16403 blk 0
rmgr: Btree len (rec/tot): 64/ 64, tx: 767, lsn: 0/01C29B78, prev 0/01C29B18, desc: INSERT_LEAF off: 1, blkref #0: rel 1663/16388/16403 blk 1
rmgr: Transaction len (rec/tot): 34/ 34, tx: 767, lsn: 0/01C29B88, prev 0/01C29B78, desc: COMMIT 2026-04-19 13:19:03.566123 CEST
```

Slika 55 - Filtriran *WAL* segment

Sa slike 55 se može primetiti da je prilikom jednostavne operacije kao što je upis jednog reda u tabelu generisan veliki broj *XLOG* zapisa. Što se tiče samih podataka, od značaja su zapisi čiji je *resource manager* *HEAP* ili *Btree*. Može se primetiti da postoji 1 zapis koji pripada *HEAP*-u i dva koja pripadaju *Btree resource manager*-u. To ispunjava naša očekivanja, obzirom da je *tuple* upisan u jednu stranicu koja pripada tabeli, upisan je u indeks i izmenjena je stranica sa metapodacima indeksa. Možemo detaljnije da pogledamo zapis koji je generisao *HEAP manager*.

9

select \* from pg\_get\_wal\_record\_info('0/1C29A98');

Data Output

Messages

Notifications

Slika 56 - Prikaz *XLOG* zapisa koji je generisao *HEAP resource manager*

Kao što se može primetiti sa slike 56. tip ovog zapisa je *INSERT+INIT* i radi se o specijalnom tipu koji se kreira kada se red prvi put upise u tabelu. Izvršen je još jedan *INSERT*, a rezultati izvršenja i proizvedeni *XLOG* zapisi prikazani su na slikama 57 i 58.

```
wal_test=# select pg_current_wal_lsn();
pg_current_wal_lsn
-----
0/1C4E248
(1 row)

wal_test=# insert into app_user values (2, 'Pera', 'panda');
INSERT 0 1
wal_test=# select pg_current_wal_lsn();
pg_current_wal_lsn
-----
0/1C4E3A0
(1 row)
```

Slika 57 - Druga *INSERT* operacija u tabelu

```
lule@lule-Asus-Vivobook:~$ sudo -u postgres /usr/lib/postgresql/18/bin/pg_waldump -p /var/lib/postgresql/18/main/pg_wal -s 0/1C4E248 -e 0/1C4E3A0
rmgr: Heap len (rec/tot): 54/ 166, tx: 768, lsn: 0/01C4E248, prev 0/01C4E210, desc: INSERT off: 2, flags: 0x00, blkref #0: rel 1663/16388/16396 blk 0 FPW
rmgr: Btree len (rec/tot): 53/ 133, tx: 768, lsn: 0/01C4E2F0, prev 0/01C4E248, desc: INSERT_LEAF off: 2, blkref #0: rel 1663/16388/16403 blk 1 FPW
rmgr: Transaction len (rec/tot): 34/ 34, tx: 768, lsn: 0/01C4E378, prev 0/01C4E2F0, desc: COMMIT 2026-04-19 13:54:20.000173 CEST
```

Slika 58 - *XLOG* zapisi proizvedeni prilikom druge *INSERT* operacije

Sa slike 58 se može videti da je prilikom ove *INSERT* operacije takođe proizveden *full page XLOG* zapis, što se može prepoznati po *FPW* skraćenici koja stoji uz ovaj zapis. Binarni sadržaj ovog zapisa u HEX formatu može se videti na slici 59. Jasno se može primetiti da se u ovom zapisu nalaze i podaci koji su upisani tokom prve *INSERT* operacije.

```
lule@lule-Asus-Vivobook:~$ sudo dd if=/var/lib/postgresql/18/main/pg_wal/000000010000000000000001 bs=1 skip=12902984 count=166 status=none | hexdump -C
00000000 a6 00 00 00 00 03 00 00 10 e2 c4 01 00 00 00 00 |.....|
00000010 00 0a 00 00 8e 97 cb dc 00 10 00 00 70 00 20 00 |.....p. |
00000020 03 7f 06 00 00 04 40 00 00 0c 40 00 00 00 00 00 |.....@...|
00000030 00 ff 03 00 00 00 00 e0 9a c2 01 00 00 00 00 20 |.....|
00000040 00 b0 1f 00 20 04 20 00 00 00 00 d8 9f 4e 00 b0 |....N...|
00000050 9f 4e 00 00 03 00 00 00 00 00 00 00 00 00 00 00 |.N.....|
00000060 00 00 00 02 00 03 00 02 08 18 00 02 00 00 00 0b |.....|
00000070 50 65 72 61 0d 70 61 6e 64 61 00 ff 02 00 00 00 |Pera.panda....|
00000080 00 00 00 00 00 00 00 00 00 00 00 01 00 03 00 02 |.....|
00000090 08 18 00 01 00 00 00 0b 4c 75 6b 61 0d 7a 65 62 |.....Luka.zeb|
000000a0 72 61 00 02 00 00 |ra....|
000000a6
```

Slika 59 - Sadržaj *INSERT XLOG* zapisa prouzrokovanog od strane *INSERT* operacije

Ovaj deo testa, kao i pokušaj teorijskog opsia sadržaja kako segmentnih fajlova, tako i pojedinačnih zapisa u fajlovima pokazuju svu kompleksnost koje PostgreSQL skriva od kako programera, tako i administratora kroz jednostavan SQL interfejs i par desetina podešavanja.

### 6.5.2 Uticaj checkpoint-a i full-page XLOG zapisa na WAL i I/O

Ideja ovog eksperimenta jeste da se prikaže uticaj učestalosti *checkpoint* operacije i parametra *full\_page\_writes* na količinu generisanog *WAL-a* i I/O. Eksperiment se sastoji od četiri testa dobijena kombinovanjem dve politike *checkpoint-a* (prikazane na slici 61) sa dva stanja parametra *full\_page\_writes* (uključen i isključen). U svim testovima koristi se ista tabela i isti skup operacija nad podacima, kako bi jedina promenljiva bila *WAL* konfiguracija. Tabela je dimenzionisana tako da se prostire na veliki broj *heap* stranica, jer se kod *full\_page\_writes* dodatni *WAL* zapis javlja po prvi put modifikovanoj stranici nakon *checkpoint-a*, a ne po SQL naredbi. Na taj način moguće je posmatrati ne samo pojedinačne efekte *checkpoint* učestalosti i *full-page* zapisa, već i njihovu međuzavisnost. Kako bi se olakšalo testiranje vreme *checkpoint-a* će biti simulirano eksplicitnim pozivom *checkpoint* komande nad bazom. U jednoj grupi testova to će biti nakon svake *update* operacije, a u drugoj nakon svih operacija.

```

create table wal_test (
    id bigint primary key,
    payload text
);

insert into wal_test
select g, repeat('x', 2000)
from generate_series(1, 200000) as g;

update wal_test
set payload = repeat('y', 2000)
where id % 2 = 0;

```

Slika 60 - Tabela i operacije korišćene u testu

```

-- CP1
checkpoint_timeout = '1min'
max_wal_size = '256MB'
checkpoint_completion_target = 0.9

-- CP2
checkpoint_timeout = '15min'
max_wal_size = '4GB'
checkpoint_completion_target = 0.9

```

Slika 61 - Podešavanja *checkpoint* operacije

Tabela 3 - Rezultati testiranja

| Checkpoint ponašanje                 | full_page_write | Količina generisanog WAL-a | Broj fsync poziva |
|--------------------------------------|-----------------|----------------------------|-------------------|
| Nakon svakog update-a                | On              | 204.05 MB                  | 78                |
| Jedan checkpoint nakon svih update-a | On              | 134.83 MB                  | 19                |
| Nakon svakog update-a                | Off             | 119.75 MB                  | 43                |
| Jedan checkpoint nakon svih update-a | Off             | 115.48 MB                  | 7                 |

Količina generisanog WAL-a merena je korišćenjem funkcija *pg\_current\_wal\_lsn* i *pg\_wal\_lsn\_diff*, dok je broj *fsync* poziva dobijen korišćenjem sistemskog pogleda *pg\_stat\_io*.

Sa stanovišta količine generisanog WAL-a rezultati pokazuju očekivano ponašanje. Najpre se može konstatovati kako se najviše WAL-a generiše kada je *full\_page\_write* uključen, a nakon svake *update* operacije se izvršava *checkpoint*. Ovo je posledica činjenice da se nakon svake *update* operacije koja se izvrši sve modifikovane *heap* stranice upisuju u WAL. Poređenje testova kada je *full\_page\_write* isključen omogućava formiranje interesantnog zaključka – *checkpoint* operacija u ovom slučaju nije previše skupa obzirom da se generisalo manje od 1MB po *checkpoint-u* u slučaju kada su *checkpoint-i* vršeni nakon svake *update* operacije.

Sa stanovišta I/O, broj *fsync* poziva proporcionalan je broju *checkpoint* operacija. Preciznije, svaka *checkpoint* operacija forsira da se *dirty* stranice iz *shared buffers* upišu na disk, pa je zato izdat veći broj *fsync* sistemskih poziva nego u slučaju kada je izvršen samo jedan *checkpoint* nakon svih operacija. Pored toga kada je *full\_page\_write* uključen, generiše se veći broj ovih poziva prilikom upisa u WAL.

Eksperiment jasno pokazuje kompromis koji se dobija u zavisnosti od podešavanja ovih parametara. *Full\_page\_write* dovodi do generisanja značajno veće količine WAL-a, ali pruža zaštitu od *corrupted heap* stranica. Sa druge strane, *checkpoint* operacija dovodi do uvećane upotrebe *fsync* sistemskog poziva koji degradira performanske u pogledu I/O operacija, ali omogućava efikasnije upravljanje WAL segment fajlovima.

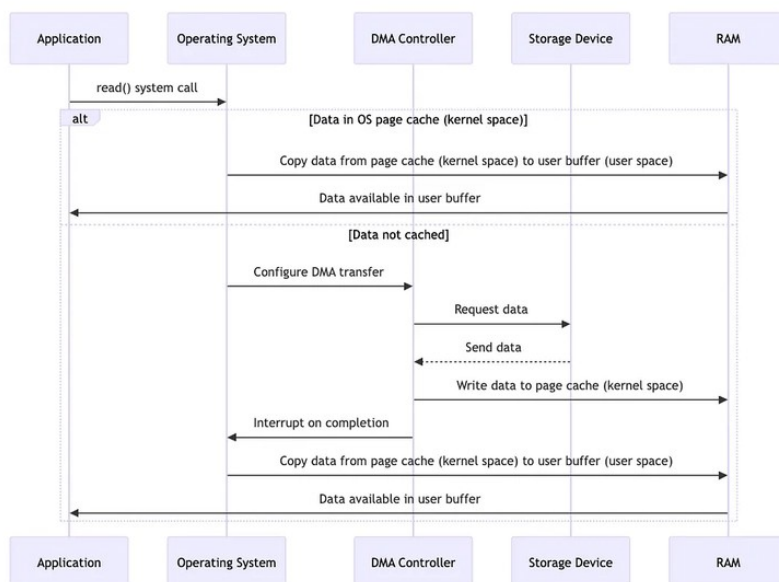
## 7 Asinhroni I/O

U dosadašnjem delu rada opisani su različiti aspekti *storage engine-a* koji je implementiran u PostgreSQL-u. Jedan od aspekata o kome je bilo malo reči je interakcija sa operativnim sistemom, odnosno pristup I/O operacijama. Pored toga poznato je da je PostgreSQL *open source* projekat i, iako postoje različite *enterprise* kompanije (Neon, EnterpriseDB...) koje aktivno učestvuju u razvoju baze, količina sredstava koja se ulažu u razvoj *open source* dela je ograničena. Ograničenja su se tokom godina najviše odrazila na razvoj I/O aspekta baze. [25] Od 2020. godine aktivno se krenulo u razmatranje pristupa kako bi se poboljšao I/O u PostgreSQL-u, a kao posledica toga, od verzije 18 PostgreSQL nudi podršku za asinhroni I/O.

U ovom poglavlju najpre će biti razmatrane prednosti asinhronog i direktnog I/O, bez zalaženja u detalje implementacije. Nakon toga će biti dat kratak opis *io\_uring* interfejsa, koji nudi Linux kernel za asinhroni I/O, a na koji se PostgreSQL implementacija AIO-a oslanja. Konačno, biće prikazan glavni benefit koji se dobija zahvaljujući asinhronom I/O.

### 7.1 Asinhroni i direktni I/O

Tradicionalni pristup I/O operacijama prolazi kroz nekoliko nivoa upravljanja, od aplikacije preko operativnog sistema, DMA kontrolera do samog RAM-a. Na slici 62 prikazan je tok pristupa različitim komponentama kada se izvrši jednostavna operacija čitanja.



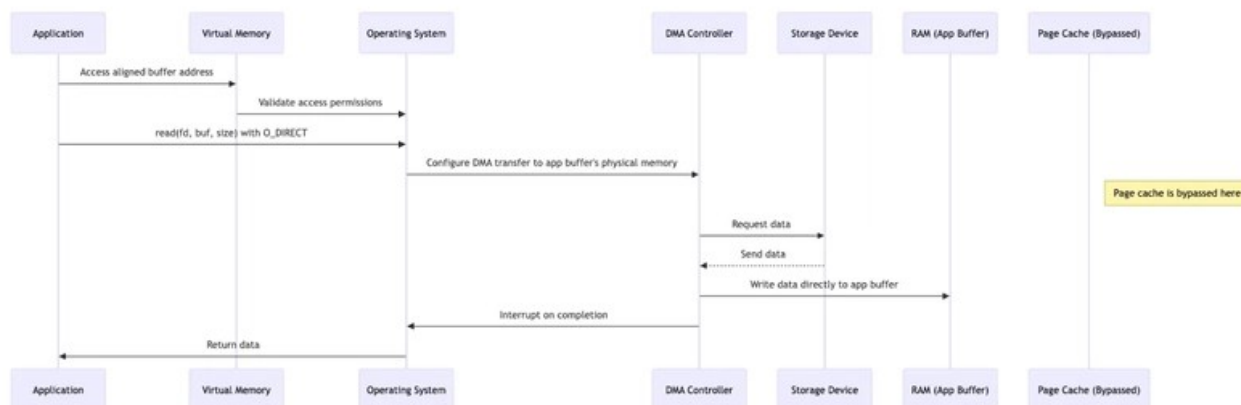
Slika 62 - Koraci prilikom čitanja podataka iz memorije, preuzeto iz [26]

Kada se iz aplikacije uputi *read* sistemski poziv, operativni sistem najpre proverava da li je stranica koja sadrži traženi podataka keširana u glavnoj memoriji (delu koji pripada kernelu – *kernel space*). Ukoliko jeste, stranice se kopira u deo memorije koji pripada korisničkom procesu (*user space*).

Ukoliko stranica nije keširana u *kernel space-u*, operativni sistem izdaje zahtev DMA kontroleru za čitanje stranice. Zbog principa prostorne i vremenske lokalnosti, prilikom čitanja stranice sa diska ne čita se samo ta jedna stranica, već se čita i nekoliko susednih stranica. Kada se podaci sa

diska učitaju u glavnu memoriju DMA kontroler generiše prekid i obaveštava operativni sistem o uspešnom čitanju podataka, nakon čega se podaci kopiraju u *user space*.

Direct I/O omogućava da se podaci sa diska u *user space* i obrnuto, pri čemu se preskače keš operativnog sistema. Na taj način se izbegava dvostruko kopiranje podataka i smanjuje se CPU *overhead*. Tok pristupa komponentama koje učestvuju u direct I/O prikazan je na slici 63.



Slika 63 - Koraci prilikom pristupa memoriji radi čitanja korišćenjem Direct I/O, preuzeto iz [26]

U slučaju PostgreSQL-a, prednosti koje se dobijaju korišćenjem Direct I/O su:

- Niži CPU *overhead* i veća propusnost – konkretno kod modernih *storage* uređaja, potreba da CPU bude uključen u kopiranje podataka iz *kernel space*-a u *user space* predstavlja usko grlo. Korišćenjem Direct I/O omogućava se da se podaci sa diska kopiraju direktno u *shared buffers* u PostgreSQL-u.
- Bolja kontrola tajminga i tempa *flush-ovanja* na disk.
- Konkurentan upis u WAL (korišćenjem *O\_DIRECT* i *O\_DSYNC* flagova) [27]

Asinhroni I/O pristup podrazumeva da aplikacija može da izda zahtev za upis/čitanje podataka na disk i da nakon toga nastavi izvršavanje bez čekanja potvrde da je pristup disku upsešno izvršen. U velikom broju *use case*-eva standardni sinhroni pristup disku je dovoljno dobar. Deo Postgres-a koji je najviše „patio“ zbog nedostatka asinhronog pristupa disku je *prefetching*. *Prefetching* je mehanizam koji PostgreSQL koristi kako bi unapred zahtevao od PostgreSQL-a da pribavi pojedine stranice (data stranice ili index stranice). Kod sinhronog pristupa disku, ovaj mehanizam je bio samo *hint* operativnom sistemu da odgovarajuću stranicu pribavi u svoj keš, ali ne i u *shared buffers* u PostgreSQL-u. Nakon toga je svakako morao biti izdat sistemski poziv za pribavljanje tih podataka. Korišćenjem kombinacije asinhronog i direktnog pristupa disku dobija se predvidljivo vreme pristupa i veća propusnost. [28]

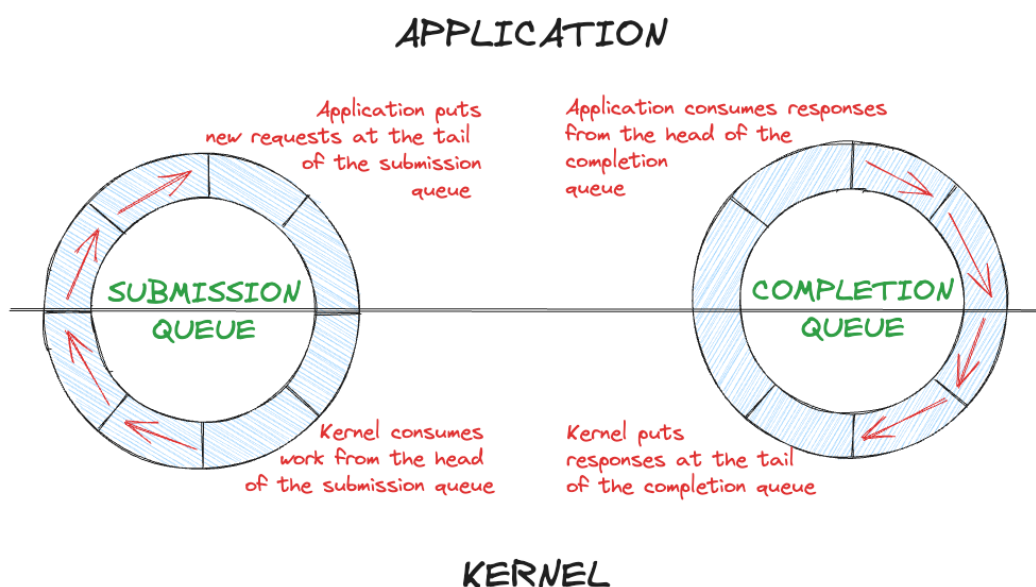
## 7.2 io\_uring

*io\_uring* je novi asinhroni I/O API za Linux, kreiran od strane Jens Axboe-a. Osnovni cilj ovog projekta je da se obezbedi API koji nema ograničenja koja nude trenutni mehanizmi za implementaciju asinhronog I/O, kao što su *select*, *poll*, *epoll* i *aio* grupe sistemskih poziva. Osnovni nedostatak koji postoji kod pomenutih mehanizama je neuniformno tretiranje fajlova deskriptora i



socket-a, podrška samo za *unbuffered* režim rada i postojanje drugih („skrivenih“) sistemskih poziva koji ovi mehanizmi skrivaju. [29]

*io\_uring* je ime dobio po strukturi podataka koji koristi za implementaciju *kernel-user space* komunikacije – *ring buffer*. Interfejs se trudi da minimizuje broj sistemskih poziva koji se koriste. Mehanizam se sastoji od 2 *ring buffer-a*, jedan koji služi za submitovanje zahteva (*submit queue* ili *SQ*) i drugi koji služi kako bi se primala obaveštenja o tome kada se neki od tih zahteva ispuni (*completion queue* ili *CQ*). Ovi *ring buffer-i* se inicijalizuju korišćenjem *io\_uring\_setup* poziva, a nakon toga se mapiraju u korisnički prostor korišćenjem *mmap* sistemskog poziva. Novi zahtevi (čitanje ili upis u fajl, mrežni zahtevi itd.) se upisuju na kraj *SQ-a*. Nakon upisa koristi se sistemski poziv *io\_uring\_enter* kako bi se kernel obavestio o novom zahtevu. Moguće je upisati nekoliko zahteva u *SQ* pre nego što se izda *io\_uring\_enter* sistemski poziv. Obaveštenja se čitaju iz *CQ* i za svaki submitovani zahtev u *SQ* postoji jedan ulaz u *CQ* koji nosi informaciju o statusu tog zahteva. [29] Na slici 64 grafički je prikazan ovaj proces.



Slika 64 - Grafički prikaz *io\_uring* mehanizma

Najznačajnije prednosti koje se dobijaju korišćenjem ovog mehanizma su:

- zero-copy sistem – kopiranje podataka između *kernel* i *user space-a* se značajno smanjuje obzirom da se sva komunikacija obavlja preko redova.
- smanjen broj sistemskih poziva – kao što je pomenuto ranije, *io\_uring* omogućava da se veći broj zahteva upiše u red pre nego što se kernel obavesti da zahtevi postoje. *io\_uring* podržava i *polling* mehanizam, gde kernel povremeno proverava sadržaj redova, bez ikakve potrebe za sistemskim pozivima. [29]

### 7.3 Asinhroni I/O u PostgreSQL-u

PostgreSQL, od verzije 18 nudi novi konfiguracioni parametar – *io\_method* koji kontroliše da li će se *read* operacije izvršavati sinhrono ili asinhrono. U aktuelnoj verziji *write* operacije su još uvek u principu sinhrono, ali obzirom da se *flush-ovanje dirty* stranice iz *shared buffers* na disk obavlja korišćenjem pozadinskih procesa pa se još uvek razmatra kako asinhrono operacije mogu da donesu poboljšanje ovom procesu.

Postoje tri moguće vrednosti koje *io\_method* parametar može da ima:

- *sync* – tradicionalno ponašanje, ono koje je korišćeno pre verzije 18. Za *prefetching* se koristi *posix\_fadvise* sistemski poziv
- *worker* – ovim podešavanjem se pokreću pozadinski *worker* procesi koji su zaduženi za pribavljanje podataka nezavisno od izvršenja upita. Glavni proces koji izvršava upit šalje zaheve ovim pomoćnim procesima i ne blokira se. Podrazumevano postoje 3 *worker* procesa
- *io\_uring* – koristi *io\_uring* interfejs u Linuksu, što znači da je ova vrednost dostupna samo pod Linux operativnim sistemom i samo ako je PostgreSQL kompajliran sa podrškom za *io\_uring*.

Jedan od „nedostataka“ koji se javlja kao posledica korišćenja asinhronih I/O operacija je monitoring. Kod sinhroniz operacija standardni alati kao što je *pg\_analyze* su dovoljno pouzdani u otkrivanju blokiranja procesa usred čekanja podataka sa diska. Kod asinhronih operacija koje koriste *worker-e*, ovo se može preduprediti tako što se vrši posmatranje upravo tih *worker* procesa, radi otkrivanja delay-a prilikom čitanja sa diska. Kod *io\_uring-a* situacija je komplikovanija jer se *read* operacije submituju direktno kernelu potpuno asinhrono. Zbog toga je uveden novi view *pg\_aio*s koji omogućava uvid u interno stanje IO operacija. [28]

## 7.4 Praktični prikaz

Za potrebe ovog praktičnog prikaza koristiće se tabela prikazana na slici 65. Za generisanje podataka korišćen je upit sa slike 66, a osnovna ideja je bila da se tabela popuni sa puno više podataka nego što je veličina *shared buffers-a* kako bi disk I/O došao do izražaja. Veličina *shared buffers-a* je podrazumevana, 128KB, dok je količina generisanih podataka oko 880MB.

```
create table app_user (  
    id                serial primary key,  
    name              varchar(20) not null,  
    password           text,  
    registration_date timestamp default now()  
);
```

Slika 65 - Tabela korišćena za praktičnu demonstraciju asinhronog I/O

```
insert into app_user (name, password)  
select  
    'Name' || i,  
    md5(random()::text)  
from generate_series(1, 10000000) as g(i);
```

Slika 66 - Upit korišćen za popunjavanje tabele sa slike 65

Upiti koji će se izvršavati nad datom tabelom i generisanim podacima su

- 1) `select count(*), max(registration_date) from app_user where id > 500;`
- 2) `select id, repeat(md5(password), 3) from app_user where id > 2500000;`

Ovi upiti će biti izvršeni dva puta (jednom sa praznim OS kešom i praznim *shared buffers*), a drugi put sa punim OS kešom i punim *shared buffers* za sva tri podešavanja parametra *io\_method*.

*Tabela 4 - Vremena izvršenja upita pod različitim io\_method-ama*

| io_method | Upit 1 – cold read | Upit 1 – warm read | Upit 2 - cold read | Upit 2 – warm read |
|-----------|--------------------|--------------------|--------------------|--------------------|
| io_uring  | 929 ms             | 595.71 ms          | 7.8 s              | 7.62 s             |
| worker    | 1055 ms            | 527.34 ms          | 7.96 s             | 7.53 s             |
| sync      | 1350 ms            | 614.24 ms          | 7.81 s             | 7.28 s             |

*Tabela 5 - Detaljnija statistika izvršavanja upita 1*

| io_method | Upit               | Cache hits | Cache misses | I/O blocking time |
|-----------|--------------------|------------|--------------|-------------------|
| io_uring  | Upit 1 – cold read | 0          | 112_582      | 153.38 ms         |
|           | Upit 1 – warm read | 846        | 111_736      | 136.2 ms          |
| worker    | Upit 1 – cold read | 0          | 112_582      | 676 ms            |
|           | Upit 1 – warm read | 1128       | 114_454      | 5.93 ms           |
| sync      | Upit 1 – cold read | 0          | 112_582      | 1574.76 ms        |
|           | Upit 1 - warm read | 564        | 112_018      | 283.46 ms         |

*Tabela 6 - Detaljnija statistika izvršavanja upita 2*

| io_method | Upit               | Cache hits | Cache misses | I/O blocking time |
|-----------|--------------------|------------|--------------|-------------------|
| io_uring  | Upit 2 – cold read | 0          | 112_582      | 127 ms            |
|           | Upit 2 – warm read | 94         | 112_582      | 128 ms            |
| worker    | Upit 2 – cold read | 0          | 112_582      | 66.67 ms          |
|           | Upit 2 – warm read | 94         | 112_488      | 4.6 ms            |
| sync      | Upit 2 – cold read | 0          | 112_582      | 366.8 ms          |
|           | Upit 2 - warm read | 94         | 112_488      | 234.31 ms         |

Uloga upita 1 je bila da se demonstriraju disk I/O performanse. Upit radi jednostavnu agregaciju, ali filter nije velike selektivnosti obzirom da filtrira samo 0.005% redova iz tabele za testiranje. Merenje performansi i praćenje statistike prilikom izvršavanja ovog upita pokazuje ono što je pomenuto u teorijskom delu ovog poglavlja – korišćenje asinhronog I/O kod *disk-heavy* operacija donosi značajno ubrzanje u izvršenju upita. Važnije od toga, vreme koje se provede čekajući disk operacije je 2-3x manje u odnosu na sinhroni pristup i, u slučaju *io\_uring-a*, izuzetno predvidivo.

Uloga upita 2 je bila demonstracija kada asinhroni pristup nema puno uticaja na ukupno vreme izvršenja. Upit 2 je *CPU-heavy* i zbog toga, iako asinhroni pristup daje značajno bolje vreme pristupa disku, generalno vreme izvršenja najvećim delom zavisi od CPU, a ne I/O vremena.



## 8 Literatura

- [1] Jeyaram Ayyalusamy, “The Internal Structure of PostgreSQL: A Deep Dive into How PostgreSQL Organizes Data,” *Medium*, Jun. 2025. <https://medium.com/@jramcloud1/the-internal-structure-of-postgresql-a-deep-dive-into-how-postgresql-organizes-data-7a0952ec0569>
- [2] “22.6. Tablespaces,” *PostgreSQL Documentation*, Nov. 21, 2024. <https://www.postgresql.org/docs/current/manage-ag-tablespaces.html>
- [3] “65.1. Database File Layout,” *PostgreSQL Documentation*, Nov. 21, 2024. <https://www.postgresql.org/docs/current/storage-file-layout.html>
- [4] PGCon, “Pluggable Table Storage in PostgreSQL - Andres Anarazel,” *YouTube*, Jun. 10, 2019. <https://www.youtube.com/watch?v=mTfvA9EQIz8> (accessed Apr. 11, 2026).
- [5] “65.6. Database Page Layout,” *PostgreSQL Documentation*, 20 Feb. 2025, [www.postgresql.org/docs/current/storage-page-layout.html](https://www.postgresql.org/docs/current/storage-page-layout.html).
- [6] Amir. “How PostgreSQL Stores Data: Heaps, Blocks, and Ctids Explained.” *Medium*, 16 Nov. 2024, [medium.com/@Amir\\_M4A/how-postgresql-stores-data-heaps-blocks-and-ctids-explained-f66d052cf87d](https://medium.com/@Amir_M4A/how-postgresql-stores-data-heaps-blocks-and-ctids-explained-f66d052cf87d). Accessed 11 Apr. 2026.
- [7] postgres. “Postgres/Src/Include/Storage/Bufpage.h at Master · Postgres/Postgres.” *GitHub*, 2025, [github.com/postgres/postgres/blob/master/src/include/storage/bufpage.h](https://github.com/postgres/postgres/blob/master/src/include/storage/bufpage.h). Accessed 11 Apr. 2026.
- [8] Suzuki, Hironobu. “1.3. Internal Layout of a Heap Table File :: Hironobu SUZUKI @ InterDB.” *Interdb.jp*, 2024, [www.interdb.jp/pg/pgsql01/03.html](http://www.interdb.jp/pg/pgsql01/03.html). Accessed 11 Apr. 2026.
- [9] Do, Harry. “Part 2 - MySQL vs PostgreSQL: Storage Architecture.” *DEV Community*, 14 Oct. 2025, [dev.to/harry\\_do/part-2-mysql-vs-postgresql-storage-architecture-2ki1](https://dev.to/harry_do/part-2-mysql-vs-postgresql-storage-architecture-2ki1). Accessed 11 Apr. 2026.
- [10] postgres. “Postgres/Src/Include/Access/Htup\_details.h at Master · Postgres/Postgres.” *GitHub*, 2025, [github.com/postgres/postgres/blob/master/src/include/access/htup\\_details.h](https://github.com/postgres/postgres/blob/master/src/include/access/htup_details.h). Accessed 12 Apr. 2026.
- [11] “66.7. Heap-Only Tuples (HOT).” *PostgreSQL Documentation*, 26 Feb. 2026, [www.postgresql.org/docs/current/storage-hot.html](https://www.postgresql.org/docs/current/storage-hot.html). Accessed 12 Apr. 2026.
- [12] Suzuki, Hironobu. “7.1. Heap Only Tuple (HOT) :: Hironobu SUZUKI @ InterDB.” *Interdb.jp*, 2024, [www.interdb.jp/pg/pgsql07/01.html](http://www.interdb.jp/pg/pgsql07/01.html). Accessed 12 Apr. 2026.
- [13] Denis Magda. “PostgreSQL Internals in Action: Shared Memory and Buffers.” *YouTube*, 4 Apr. 2024, [www.youtube.com/watch?v=S0LEDGAnn8](https://www.youtube.com/watch?v=S0LEDGAnn8). Accessed 12 Apr. 2026.
- [14] priteshsurana. “PostgreSQL Internals: Inside the Storage Engine.” *DEV Community*, 5 Apr. 2026, [dev.to/priteshsurana/postgresql-internals-inside-the-storage-engine-1bhm](https://dev.to/priteshsurana/postgresql-internals-inside-the-storage-engine-1bhm). Accessed 12 Apr. 2026.
- [15] Mehman Jafarov. “Process and Memory Architecture of PostgreSQL — Part 2.” *Medium*, 8 Jan. 2025, [medium.com/@mehmanjafarov1905/process-and-memory-architecture-of-postgresql-part-2-abc2dad16b0e#c293](https://medium.com/@mehmanjafarov1905/process-and-memory-architecture-of-postgresql-part-2-abc2dad16b0e#c293). Accessed 12 Apr. 2026.

[16] Walia, Aryan. "PostgreSQL Write Path Explained: Fsync, WAL, COPY, and Unlogged Tables." *Medium*, 12 Jan. 2026, [medium.com/@ariyanwaliya/postgresql-write-path-explained-fsync-wal-copy-and-unlogged-tables-8f4243e42292](https://medium.com/@ariyanwaliya/postgresql-write-path-explained-fsync-wal-copy-and-unlogged-tables-8f4243e42292). Accessed 12 Apr. 2026.

[17] Li, Dichen. "30 Years of PostgreSQL Buffer Manager Locking Design Evolution." *Medium*, 26 Mar. 2025, [medium.com/@dichenldc/30-years-of-postgresql-buffer-manager-locking-design-evolution-e6e861d7072f](https://medium.com/@dichenldc/30-years-of-postgresql-buffer-manager-locking-design-evolution-e6e861d7072f). Accessed 16 Apr. 2026.

[18] Suzuki, Hironobu. "8. Buffer Manager :: Hironobu SUZUKI @ InterDB." *Interdb.jp*, 2024, [www.interdb.jp/pg/pgsql08/index.html](http://www.interdb.jp/pg/pgsql08/index.html). Accessed 18 Apr. 2026.

[19] Mohamed-Ali. "Database Logging: WAL, Redo, and Undo Mechanisms - Mohamed-Ali - Medium." *Medium*, 16 May 2025, [medium.com/@moali314/database-logging-wal-redo-and-undo-mechanisms-58c076fbe36e](https://medium.com/@moali314/database-logging-wal-redo-and-undo-mechanisms-58c076fbe36e).

[20] "30.3. Write-Ahead Logging (WAL)." *PostgreSQL Documentation*, 13 Oct. 2022, [www.postgresql.org/docs/current/wal-intro.html](http://www.postgresql.org/docs/current/wal-intro.html).

[21] Suzuki, Hironobu. "9. Write Ahead Logging (WAL) :: Hironobu SUZUKI @ InterDB." *Interdb.jp*, 2024, [www.interdb.jp/pg/pgsql09/index.html](http://www.interdb.jp/pg/pgsql09/index.html). Accessed 18 Apr. 2026.

[22] postgres. "Postgres/Src/Include/Access/Xlogrecord.h at Master · Postgres/Postgres." *GitHub*, 2025, [github.com/postgres/postgres/blob/master/src/include/access/xlogrecord.h](https://github.com/postgres/postgres/blob/master/src/include/access/xlogrecord.h). Accessed 18 Apr. 2026.

[23] "How to Understand the WAL Configuration-Related Parameters in PostgreSQL." *Cybrosys*, 2026, [www.cybrosys.com/research-and-development/postgres/how-to-understand-the-wal-configuration-related-parameters-in-postgresql](http://www.cybrosys.com/research-and-development/postgres/how-to-understand-the-wal-configuration-related-parameters-in-postgresql). Accessed 19 Apr. 2026.

[24] "28.5. WAL Configuration." *PostgreSQL Documentation*, 26 Feb. 2026, [www.postgresql.org/docs/current/wal-configuration.html](http://www.postgresql.org/docs/current/wal-configuration.html). Accessed 19 Apr. 2026.

[25] Corbet, Jonathan. "Re: Why We Are Going to Have to Go DirectIO." *PostgreSQL Mailing List Archives*, 4 Dec. 2013, [www.postgresql.org/message-id/20131204083345.31c60dd1%40lwn.net](http://www.postgresql.org/message-id/20131204083345.31c60dd1%40lwn.net). Accessed 19 Apr. 2026.

[26] Sachin Tolay. "Traditional IO vs Mmap vs Direct IO: How Disk Access Really Works." *DEV Community*, 20 July 2025, [dev.to/sachin\\_tolay\\_052a7e539e57/traditional-io-vs-mmap-vs-direct-io-how-disk-access-really-works-1h4l](https://dev.to/sachin_tolay_052a7e539e57/traditional-io-vs-mmap-vs-direct-io-how-disk-access-really-works-1h4l). Accessed 29 Apr. 2026.

[27] Freund, Andres. "Asynchronous and "Direct" IO Support for PostgreSQL." *PostgreSQL Mailing List Archives*, 23 Feb. 2021, [www.postgresql.org/message-id/20210223100344.llw5an2aklengrmn@alap3.anarazel.de](http://www.postgresql.org/message-id/20210223100344.llw5an2aklengrmn@alap3.anarazel.de). Accessed 29 Apr. 2026.

[28] Fittl, Lukas. "Waiting for Postgres 18: Accelerating Disk Reads with Asynchronous I/O." *Pganalyze*, 7 May 2025, [pganalyze.com/blog/postgres-18-async-io](https://pganalyze.com/blog/postgres-18-async-io).

[29] "What Is Io\_uring? — Lord of the Io\_uring Documentation." *Unixism.net*, 2020, [unixism.net/loti/what\\_is\\_io\\_uring.html](http://unixism.net/loti/what_is_io_uring.html). Accessed 29 Apr. 2026.